A FIELD GUIDE FOR THE AI BUILDER

Agentic Atlas Q&A.

Explore how AI agents think, act, and work together. From your first concept to the systems you build next.

34	93	320	3 levels
Knowledge categories	Topics to explore	Questions. Real answers.	Basic → Intermediate → Advanced

What this edition covers. All 72 questions visible in your supplied carousel images, plus 248 clearly labeled additions. The original 100 answers are retained and expanded into 34 categories and 93 topics. Every topic has Basic, Intermediate, and Advanced questions with answers.

The images credit **bhavs.ai** and advertise a 223-question PDF. That PDF was not supplied; this document does not reproduce or claim access to its unseen questions or answers. Answers here are newly written. The taxonomy is a broad learning map, not a claim that the field has a fixed or exhaustive set of categories.

Study tip: choose one category, explain an answer out loud, then compare it with the reference. Behavioral answers are preparation frameworks, not claims about your personal experience.

CATEGORY 01 / 34

LLM fundamentals

3 topics · 13 questions

Tokens and context

Basic 2 Q&A

001 **What is a token, and why do token counts matter for agents?**

Basic

A token is a unit produced by a model's tokenizer: it may represent a word, part of a word, punctuation, or other data. Agents accumulate instructions, tool schemas, tool results, and conversation history across steps. Token counts affect context capacity, latency, and often API cost. Measure with the tokenizer or usage metrics for the model you actually use; a fixed characters-per-token rule is only an estimate.

002 **What is a context window, and what happens when you exceed it?**

Basic

The context window is the maximum sequence a model can process for a request, subject to provider-specific input and output limits. Oversized requests may be rejected, or a surrounding application may truncate or summarize them. The model does not automatically retain everything omitted from its input. Reserve space for output and upcoming tool results, and explicitly preserve task state when compressing history.

Intermediate 1 Q&A

101 **How should an agent budget context across a long task?**

Intermediate

Reserve capacity for instructions, the next tool result, and output before selecting history and evidence. Keep compact explicit task state and retrieve details on demand. Measure actual usage, including tool schemas. Drop irrelevant content deliberately rather than relying on accidental truncation.

Advanced 1 Q&A

102 **How do you evaluate information loss after context compaction?**

Advanced

Create tasks that require earlier constraints, decisions, and evidence, then compare continuation with full and compacted histories. Check omissions, false additions, and changed actions, not only summary fluency. Preserve source references so missing detail can be recovered. Use repeated trials and include adversarially similar facts.

Sampling and reliability

Basic 2 Q&A

003 **Explain temperature and top-p. When would you set temperature low?**

Basic

Temperature changes how sharply the model favors high-probability next tokens. Top-p restricts sampling to a set of tokens whose cumulative probability reaches a threshold. Low temperature is often useful for extraction, classification, and consistent formatting. Neither setting guarantees correctness or determinism. Some models constrain or do not expose these parameters, so check the chosen API and evaluate rather than assuming universal settings.

005 **What causes hallucinations, and how do you reduce them in agents?**

Basic

Models generate plausible continuations rather than checking truth automatically. Missing evidence, ambiguous instructions, unreliable retrieval, and unsupported inference can produce confident errors. Ground factual claims in trusted sources, require evidence for important claims, validate tool outputs, and let the system abstain when information is missing. Evaluate factuality and task outcomes; asking the model to be accurate is not sufficient.

Intermediate 1 Q&A

004 **Why can LLM outputs vary even at temperature 0?**

Intermediate

Temperature 0 usually means selecting a highest-scoring token, but tiny numerical differences, distributed execution, changes to model versions, or provider implementation details can alter close decisions. Retrieval results, tools, and hidden application state can also change the effective input. Pin supported model versions, record complete inputs and tool results, and judge reproducibility statistically instead of assuming byte-identical output.

Advanced 1 Q&A

103 **How do you measure uncertainty without trusting a model's confidence claim?**

Advanced

Use observed performance on representative labeled tasks, calibration experiments when probabilities are available, and evidence checks suited to the decision. Evaluate abstention and escalation behavior. A generated statement such as “95% certain” is not a calibrated probability. Tool-verified outcomes provide stronger evidence than rhetorical confidence.

Efficiency

Basic 1 Q&A

104 **What is the difference between input and output tokens?**

Basic

Input tokens encode the material sent to the model; output tokens are generated in response. Both can affect cost and capacity, with provider-specific accounting for caching or reasoning. Multi-step agents repeat this accounting across calls. Use reported usage instead of estimating the whole task from its final answer length.

Intermediate 3 Q&A

006 **What is prompt caching and how does it help agents?**

Intermediate

Prompt caching reuses computation for supported repeated prompt content, commonly a matching prefix. Stable instructions and tool definitions can make repeated agent calls cheaper or faster when the provider’s cache conditions are met. It is different from caching a final answer. Eligibility, lifetime, pricing, and matching rules vary, so inspect reported cache usage and keep dynamic content from unnecessarily disrupting reusable prefixes.

007 **How is prompt caching different from response caching?**

Intermediate

Prompt caching reuses model computation for eligible input content; a model still generates a response. Response caching returns a stored answer for an equivalent request. Cached answers can become stale or cross permission boundaries if the key omits user or tenant scope. Use explicit expiration and invalidation, and avoid replaying a cached answer when fresh external state is required.

008 **How should you choose a model for an agent task?**

Intermediate

Benchmark candidate models on realistic end-to-end tasks with the same tools, budgets, and success criteria. Measure task success, policy adherence, latency, total cost, and capability limits such as context or structured-output support. A stronger general benchmark score may not predict your specific workload. Use routing only when its additional complexity produces a measured benefit.

Advanced 1 Q&A

105 **How do you prevent caching from returning unauthorized agent results?**

Advanced

Include identity, tenant, permission-relevant scope, input version, and freshness requirements in cache design. Revalidate access before returning sensitive content and invalidate on relevant policy changes. Avoid caching side-effect execution as if it were a pure read. Test cross-user and revoked-access cases explicitly.

CATEGORY 02 / 34

Prompting and structured outputs

3 topics · 14 questions

Instructions and examples

Basic 3 Q&A

009 **What makes a good system prompt for an agent?**

Basic

Specify the goal, scope, constraints, available capabilities, tool-use rules, completion criteria, and how to handle missing information or failure. Distinguish trusted instructions from documents the agent reads. Prefer concrete rules and a few representative examples over an oversized role description. Enforce permissions and budgets in application code; a prompt is guidance, not an access-control boundary.

010 **How do you get reliable JSON out of an LLM?**

Basic

Use schema-constrained structured output when the model and API support it. Define required fields, types, enums, and null behavior, then parse and validate the result in code. Handle refusals, truncation, and unsupported schema features. Valid JSON is only syntactic success: also check semantic constraints such as a refund amount being within the order's refundable balance. Use bounded repair attempts rather than retrying forever.

011 **Zero-shot vs few-shot prompting: when do examples help?**

Basic

Zero-shot instructions describe the task without worked examples. Few-shot instructions include examples of the desired mapping from input to output. Examples help with unusual labels, precise formatting, edge cases, or a specific style. They also consume context and can bias outputs toward accidental patterns. Use diverse examples and a separate held-out evaluation set to verify that they improve performance.

Intermediate 1 Q&A

106 **How do you write examples that do not overfit one request?**

Intermediate

Vary wording, input structure, and edge conditions while preserving the intended rule. Include counterexamples for common mistakes and keep examples distinct from the held-out evaluation set. Inspect whether the model copies incidental details. Use the smallest set that improves measured task behavior.

Advanced 1 Q&A

107 **How do you evaluate prompt robustness across changing context?**

Advanced

Test the same task under irrelevant material, reordered evidence, conflicting untrusted instructions, and different context lengths. Grade execution and supported conclusions, not exact wording alone. Keep the trusted instruction boundary fixed. Analyze failures by transformation to identify which context changes break the design.

Chaining and action policies

Basic 1 Q&A

108 **What is a structured handoff between prompt stages?**

Basic

It is a defined output from one stage that becomes validated input for the next, such as extracted fields with source references. Use a schema and explicit missing-value handling. The next stage should not need to guess what an informal paragraph means. Preserve uncertainty instead of converting it into a definite fact.

Intermediate 2 Q&A

012 What is prompt chaining, and why use it?

Intermediate

Prompt chaining divides a task into explicit stages such as extract, validate, enrich, and summarize. Each stage has a defined input and output that can be checked independently. It is useful when intermediate validation improves reliability or stages need different tools or models. Extra stages add latency and failure opportunities; a simple task may be better handled by one call.

013 How do you prompt a tool-using agent differently from a chatbot?

Intermediate

Define when to use tools, what evidence to collect, how to interpret tool failures, and which actions require user confirmation or policy checks. State that tool results and retrieved documents are data rather than new authority. Define stop conditions and require reporting the observed outcome. The runtime must validate arguments and authorize actions before execution, regardless of the model's proposed call.

Advanced 1 Q&A

109 How do you avoid error amplification in prompt chains?

Advanced

Validate intermediate outputs, keep supporting evidence, and stop or recover when an upstream stage lacks required information. Evaluate each stage and the complete task because locally plausible outputs can combine into a wrong result. Avoid repeated paraphrasing that loses qualifiers. Add stages only when they improve end-to-end outcomes.

Change management

Basic 1 Q&A

110 **What belongs in a prompt release record?**

Basic

Record the instruction text, examples, schema, model configuration, tool versions, release identifier, and evaluation results. Link the change to its intended outcome. This allows later failures to be reproduced and compared. A screenshot of the final prompt is less useful than a versioned source file.

Intermediate 3 Q&A

014 **How do you version and test prompts?**

Intermediate

Store prompts, schemas, model configuration, and tool contracts in version control with an identifiable release. Run representative regression cases before deployment, including failure and adversarial cases. Compare task success, safety, cost, and latency with the previous version. Use staged rollout and rollback. Log version identifiers with each trace so a production incident can be traced to the exact configuration.

015 **How do you separate prompt changes from model changes in an experiment?**

Intermediate

Change one major factor at a time when possible, pin model versions and tool behavior, and run the same held-out cases under comparable conditions. Use repeated trials for variable tasks and report uncertainty. If multiple factors must change, use an experimental design that can separate their effects. Record complete configuration identifiers so later comparisons are reproducible.

016 **Why does valid structured output still need business validation?**

Intermediate

A schema can enforce that an amount is numeric or a status is one of several strings, but it cannot establish that a refund is authorized or a cited document supports a claim. Check relationships, ownership, ranges, and live system state in application code. Handle a well-formed but invalid result as a controlled failure rather than executing it blindly.

Advanced 1 Q&A

111 **How do you roll back a prompt change safely?**

Advanced

Restore a known configuration while accounting for in-flight runs and state-format compatibility. Determine whether external actions already occurred and reconcile affected tasks. Monitor whether the rollback fixes the targeted regression. Reverting text alone may not restore behavior if tools, models, or source data changed at the same time.

CATEGORY 03 / 34

Tool use and function calling

3 topics · 12 questions

Execution contracts

Basic 2 Q&A

017 **How does tool calling work end to end?**

Basic

The application sends the model instructions and tool schemas. The model returns a proposed tool name and arguments. Application code validates the arguments, checks permission and policy, executes the operation, and returns a correlated result. The model then continues using that result. The loop may repeat, but needs limits on steps, time, cost, and retries. Tool requests are proposals, not proof of execution.

018 **Who executes a tool: the model or your code?**

Basic

The model normally proposes a call; an application runtime, hosted tool service, or external system executes it. Even when an API offers built-in tools, execution is performed by the provider's runtime rather than by token generation itself. Keep credentials and authorization outside the prompt, and record actual tool outcomes so the assistant cannot mistake a planned action for a completed one.

Intermediate 1 Q&A

112 **What validation should happen before dispatching a tool call?**

Intermediate

Check the tool is available to the caller, parse the arguments, validate schema and business constraints, and authorize the exact resource and operation. Enforce deadlines and budgets. For writes, establish idempotency or outcome-checking behavior. Only then execute and return the actual result to the model.

Advanced 1 Q&A

113 **How do you handle a model response containing several tool calls?**

Advanced

Determine whether calls are independent or have ordering and shared-state dependencies. Validate and authorize each one separately. Parallelize only when semantics allow it, correlate every result, and handle partial success explicitly. A model emitting multiple calls does not guarantee they are safe to execute concurrently.

Tool design

Basic 1 Q&A

114 **What is a tool schema?**

Basic

A tool schema describes the arguments and types a caller may supply, often with descriptions and required fields. It helps the model construct a request and lets code reject malformed inputs. It does not prove permission, business validity, or that execution succeeded. Those checks belong in the application and service.

Intermediate 2 Q&A

019 What makes a good tool description?

Intermediate

Describe the tool's purpose, when to use it, when not to use it, argument meanings, relevant preconditions, output shape, and side effects. Use names that distinguish similar operations. State whether an operation is read-only or changes external state. A narrow, well-documented contract reduces ambiguity; executable validation and permission checks must still enforce the contract.

020 Should tools mirror API endpoints or whole tasks?

Intermediate

Prefer operations that match the agent's useful decisions while preserving visibility into consequential actions. A task-level tool can reduce fragile call sequences, but an overly broad tool may hide side effects or permissions. Keep reads and writes distinguishable, expose required review points, and make operations composable. Choose granularity using real traces and evaluation results, rather than copying every endpoint or making one all-powerful tool.

Advanced 1 Q&A

115 How do you design a high-level tool without hiding consequential actions?

Advanced

Define its exact side effects, input constraints, review requirements, and result states. Separate planning or preview from committing changes where needed. Make partial completion observable and expose stable operation identifiers. Evaluate whether reduced call complexity preserves the user's ability to understand and control the resulting action.

Failure and selection

Basic 1 Q&A

116 **What should an agent do if a required tool is unavailable?**

Basic

Explain the missing capability and which part of the task is blocked. Continue useful work that does not depend on it, such as preparing an artifact or inspecting available evidence. Do not fabricate execution or silently use a different account. Offer a concrete next step consistent with the task.

Intermediate 2 Q&A

021 **How should a tool report errors?**

Intermediate

Return a structured status with an error code, concise description, retryability, and any partial completion. Distinguish invalid arguments, permission denial, rate limits, transient outages, and an unknown outcome. Do not leak secrets in error messages. For writes, return a stable operation identifier or idempotency key status so the agent can check whether the action happened before retrying.

022 **How many tools is too many?**

Intermediate

There is no universal threshold. A tool set is too large when similar names confuse selection, schemas consume excessive context, or success degrades in evaluations. Scope tools by task and permission, retrieve relevant tools when appropriate, and consolidate redundant contracts. Measure incorrect selection, argument errors, latency, and task success before and after changes.

Advanced 1 Q&A

117 **How do you prevent retries from multiplying side effects?**

Advanced

Use a stable operation identity, enforce idempotency at the write service, and reconcile unknown outcomes before issuing a new operation. Bound retries and distinguish transient failures from invalid requests. Test crashes between external completion and local recording. Retry logic should follow operation semantics, not a universal retry-on-error rule.

CATEGORY 04 / 34

The agent loop and architecture

2 topics · 8 questions

Definitions and control flow

Basic 2 Q&A

023 **What is an AI agent, and how is it different from a workflow?**

Basic

An AI agent uses model output to choose some next actions in pursuit of a goal, usually with tools and state. A workflow has control flow that is largely prescribed by code or a graph. The distinction is a spectrum: a workflow may contain an agent step, and an agent may operate inside strict boundaries. Use the least autonomous design that meets the task's requirements.

024 **Describe the agent loop.**

Basic

Receive a goal and current state, assemble relevant context, ask the model for a response or action, validate and execute permitted tool calls, observe results, and update state. Repeat until completion, escalation, cancellation, or a budget limit. Persist enough state to explain outcomes and recover safely. A successful final message should reflect verified results, not merely the agent's intended actions.

Intermediate 1 Q&A

025 **When should you NOT use an agent?**

Intermediate

Avoid an agent when deterministic code or a fixed workflow already handles the problem reliably and cheaply, or when unpredictable choices add unacceptable risk. Examples include exact arithmetic, straightforward ETL, and a simple known API sequence. An agent is more justified when the next action depends on ambiguous information that cannot be fully specified in advance. Compare against a simpler baseline.

Advanced 1 Q&A

118 **How do you determine whether autonomy adds value to an architecture?**

Advanced

Compare a bounded agent against a deterministic or fixed-workflow baseline on the same task distribution and budget. Measure verified outcomes, recovery, user intervention, latency, and cost. Inspect which decisions truly need model judgment. Keep autonomy where its benefits exceed the added uncertainty and operational burden.

Boundaries and implementation

Basic 1 Q&A

026 **What stop conditions should every agent have?**

Basic

Include a verifiable completion condition, maximum steps, elapsed-time deadline, token or monetary budget, cancellation, and bounded retries. Add checks for repeated actions or lack of progress. A blocked dependency or action outside permission should lead to a clear escalation rather than looping. These conditions should be implemented in the runtime, with a recorded termination reason.

Intermediate 2 Q&A

027 **How do you build an agent loop without a framework?**

Intermediate

Keep explicit state containing messages, task status, counters, and tool outcomes. Call a model that can propose structured actions; dispatch calls through a registry after validation and authorization. Append results, checkpoint state, and loop under enforced limits. Handle errors and cancellation in code. Start with serial execution; introduce concurrency only when operations are independent and shared-state behavior is defined.

028 **What do frameworks like LangGraph or Agent SDKs add?**

Intermediate

Depending on the framework and version, they can provide state graphs, tool dispatch, handoffs, checkpoints, streaming, tracing, and human review hooks. They reduce repeated orchestration code but introduce abstractions and operational dependencies. Verify which durability and recovery guarantees are actually supported. Framework adoption does not remove the need for application authorization, testing, and evaluation.

Advanced 1 Q&A

119 **How should an agent runtime enforce resource budgets?**

Advanced

Track step, time, token, tool, and monetary limits in authoritative runtime state. Reserve estimated resources before dispatch where necessary, then reconcile actual usage. Include fan-out and retries. On exhaustion, checkpoint and report partial results rather than relying on the model to notice a natural-language budget instruction.

CATEGORY 05 / 34

Agent patterns and multi-agent systems

2 topics · 8 questions

Common patterns

Basic 1 Q&A

029 **Explain routing.**

Basic

Routing selects a specialized path, model, or tool set based on the task or observed state. For example, classify a support request into billing, technical troubleshooting, or account access. Routes can be deterministic or model-based. Validate the destination, provide a fallback for uncertainty, and evaluate misrouting, because a confident wrong route can quietly degrade every later step.

Intermediate 2 Q&A

030 **Explain orchestrator-workers.**

Intermediate

An orchestrator decomposes a task, assigns bounded subtasks to workers, and combines their results. Workers may be models, agents, or ordinary functions. It works well for separable research or independent code reviews. Specify input/output contracts, resource limits, shared-state ownership, and how conflicting results are resolved. Parallel workers help only when their outputs can be integrated efficiently.

031 **Explain evaluator-optimizer (reflection).**

Intermediate

A generator produces a candidate, an evaluator checks it against explicit criteria, and an optimizer revises it using the feedback. Repeat under a fixed budget until a threshold is met or progress stops. Tests or external evidence are stronger than ungrounded self-critique. A second model can repeat the first model's mistakes, so evaluate whether the extra loop actually improves outcomes.

Advanced 1 Q&A

120 **How do you choose among routing, workers, and evaluator loops?**

Advanced

Start from the task structure: routing selects a path, workers handle separable work, and evaluator loops revise against a usable criterion. Compare the smallest suitable pattern with simpler baselines under equal budgets. Composing all patterns can add overhead without benefit. Validate the integrated architecture, including handoffs and stopping rules.

Collaboration tradeoffs

Basic 1 Q&A

121 **What is the purpose of delegating a bounded subtask?**

Basic

Delegation can give a worker focused context and a clear deliverable while the parent coordinates the overall task. It helps when work is genuinely separable or needs different capabilities. Specify ownership and evidence requirements. Delegating a vague goal without an integration plan often creates more coordination work.

Intermediate 1 Q&A

034 **What is a subagent?**

Intermediate

A subagent is a delegated execution unit given a narrower task by a parent agent or orchestrator. It may have its own context, tools, and budget. Define what it can change and what evidence it must return. A subagent is not necessarily a security boundary: tool credentials and access controls determine actual isolation.

Advanced 2 Q&A

032 **When is a multi-agent system better than a single agent?**

Advanced

Use multiple agents when tasks can be separated with limited coordination, different tools or permissions are needed, or independent perspectives produce measurable quality gains. Examples include parallel evidence collection from distinct sources. Compare against a single-agent baseline under the same total cost and latency budget. More agents alone do not imply more intelligence, reliability, or speed.

033 **What are the downsides of multi-agent systems?**

Advanced

They add communication overhead, duplicated work, inconsistent assumptions, integration failures, and more complicated tracing and recovery. Shared resources can produce race conditions, and correlated model mistakes can survive apparent consensus. Use bounded roles, explicit contracts, one owner for each mutable artifact, and a clear final decision process. Track total system success rather than each worker's local success alone.

CATEGORY 06 / 34

MCP and integrations

2 topics · 9 questions

Protocol fundamentals

Basic 3 Q&A

035 **What is MCP (Model Context Protocol)?**

Basic

MCP is a protocol for connecting AI applications to external capabilities and context through a common interface. A host manages client connections to servers that expose supported capabilities. It can make a connector reusable across compatible applications. The protocol standardizes interaction, but it does not by itself decide whether a requested action is appropriate or authorized.

036 **What problem does MCP solve?**

Basic

Without a shared protocol, each AI application may need a custom integration for each data source or tool service. MCP provides common discovery and interaction patterns so compatible clients can connect to reusable servers. It reduces integration duplication but still requires deployment, identity, permission, and operational decisions. Support depends on the client, server, negotiated capabilities, and protocol revision.

037 **What are MCP's core primitives?**

Basic

The main server-exposed primitives are tools for operations, resources for contextual data, and prompts for reusable interaction templates. A protocol implementation can also negotiate additional features; their availability varies by revision and client. Keep the distinction clear: reading a resource is different from executing a tool, and an exposed prompt is not a replacement for the host's trusted instructions.

Intermediate 1 Q&A

122 **How do MCP hosts, clients, and servers relate?**

Intermediate

A host is the application using MCP capabilities; it manages client connections to servers. Servers expose supported capabilities such as tools and resources. Exact connection behavior depends on negotiated features and implementation. Keep user permissions and consent in the host and owning services rather than treating every server capability as automatically approved.

Advanced 1 Q&A

123 **How do you design MCP integration compatibility tests?**

Advanced

Test negotiation, capability discovery, request and response schemas, errors, timeouts, authentication, and permission denial against the protocol revision you support. Include server restarts and changed tool descriptions. Verify task outcomes in addition to message exchange. Avoid relying on an optional feature merely because one client happens to implement it.

Security and execution

Basic 1 Q&A

124 **Does connecting an MCP server authorize every tool it exposes?**

Basic

No. Connection establishes an available integration, subject to the host and server's authentication and capabilities. Each operation still needs appropriate user scope and authorization. Distinguish read access from actions that modify or transmit data. The model should not infer unlimited authority from a successful connection.

Intermediate 2 Q&A

038 **What are the security risks of MCP?**

Intermediate

Risks include malicious or compromised servers, misleading tool descriptions, prompt injection through returned content, credential exposure, overbroad scopes, and unintended data transfers. Authenticate servers, restrict available tools and scopes, validate arguments and results, and authorize consequential calls in the host or service. Treat descriptions and server content as potentially untrusted, and audit changes to installed integrations.

039 **When would you use MCP vs direct function calling?**

Intermediate

MCP is useful when a capability should be reusable across compatible hosts or benefits from standardized discovery and remote integration. Direct function calling can be simpler for a small application with private, tightly controlled functions. They are not mutually exclusive: a host may expose MCP server tools to a model using the model provider’s function-calling interface.

Advanced 1 Q&A

040 **What is code execution with MCP?**

Advanced

This is an architectural pattern in which an agent writes code that invokes wrappers around MCP tools inside an execution environment. Code can filter results, compose operations, and return a compact summary rather than passing every intermediate result through the model. It is not a universal guarantee of MCP. The code runner still needs isolation, scoped credentials, network limits, timeouts, and checks on side effects.

CATEGORY 07 / 34

RAG, memory and context engineering

2 topics · 10 questions

Retrieval foundations

Basic 2 Q&A

041 **What is RAG, and why use it?**

Basic

Retrieval-augmented generation fetches relevant external information and supplies it to a model before generation. It helps an application answer using private, recent, or domain-specific material without relying solely on model weights. Retrieval can be lexical, vector-based, or hybrid. RAG does not guarantee correctness: irrelevant evidence, stale documents, and unsupported synthesis still need to be detected.

042 **Describe a RAG pipeline.**

Basic

Ingest authorized documents, normalize them, split them into useful chunks, retain metadata, and build searchable indexes. At query time, retrieve candidates, optionally rerank them, assemble an evidence-limited context, and generate an answer with citations. Enforce access control before content enters the model. Evaluate retrieval coverage and answer groundedness separately so you know which stage failed.

Intermediate 1 Q&A

125 **Why should retrieval enforce permissions before model generation?**

Intermediate

Once unauthorized content enters model context, filtering the final answer cannot reliably undo exposure. Restrict candidate retrieval and document access using the authenticated user's current permissions. Keep caches and summaries scoped as well. Test both direct document reads and indirect leaks through retrieved excerpts.

Advanced 1 Q&A

126 **How do you evaluate a retrieval system for changing enterprise data?**

Advanced

Use queries with known relevant evidence, document versions, and access scopes, including recent edits and revoked permissions. Measure recall, ranking quality, freshness, and leakage. Test deletion propagation and ingestion failure. An accurate benchmark on a static public corpus does not establish safe behavior on live organizational data.

Context and memory

Basic 1 Q&A

127 **Does storing a conversation mean the model remembers it?**

Basic

No. Storage preserves data outside the active request, while recall requires selecting and supplying relevant information to the model. The application decides what history, summaries, or memory to retrieve. Preserve explicit state and provenance. Avoid implying that a model can access material omitted from its current context automatically.

Intermediate 4 Q&A

043 What is context engineering?

Intermediate

Context engineering is the design of what information an agent receives at each step: instructions, tool definitions, task state, selected history, retrieved evidence, and memory. It includes ordering, provenance, token budgeting, and removal of irrelevant content. Its purpose is to give the model sufficient reliable information for the next decision without overwhelming it or exposing data it should not access.

044 What is compaction?

Intermediate

Compaction replaces a large interaction history with a smaller representation of the information needed to continue. Preserve goals, constraints, decisions, unresolved tasks, relevant evidence references, and tool outcomes. It is lossy: a summary may omit or distort important detail. Keep original records outside the active context and retrieve them when needed; compaction does not enlarge the model's actual context window.

045 What is agentic search, and how does it differ from classic RAG?

Intermediate

A classic RAG setup often executes a predetermined retrieve-then-answer sequence. Agentic search lets the system decide whether to search again, reformulate a query, inspect a source, or pursue a missing fact. This flexibility can improve difficult research but adds cost, latency, and opportunities to drift. Limit iterations and evaluate whether evidence coverage improves over the fixed pipeline.

046 Short-term vs long-term memory in agents?

Intermediate

Short-term memory is state needed for the current task, such as active messages, tool results, and a working plan. Long-term memory persists across tasks, such as a user preference or validated domain fact. Storage outside the model enables persistence, but recall still requires retrieval into context. Use provenance, access controls, correction and deletion mechanisms, and expiration where appropriate; do not persist every observation indiscriminately.

Advanced 1 Q&A

128 **How do you choose between a larger context window and selective retrieval?**

Advanced

Compare task accuracy, evidence coverage, cost, latency, and permission handling under realistic document volumes. Larger context can simplify some tasks but still contains irrelevant or conflicting material. Retrieval reduces input but may miss evidence. Use a measured combination rather than assuming either technique removes the other's failure modes.

CATEGORY 08 / 34

Evals and observability

2 topics · 8 questions

Evaluation design

Basic 1 Q&A

047 **Why are evals essential for agents?**

Basic

Agent behavior depends on prompts, models, tools, state, and a sequence of decisions, so a few successful demonstrations do not establish reliability. Evals quantify task success, policy adherence, regressions, and operational tradeoffs across representative cases. Include failures and edge cases, not just happy paths. Combine offline evaluation with controlled production monitoring because neither alone captures every failure mode.

Intermediate 2 Q&A

048 What are the parts of an agent eval?

Intermediate

Define the task and environment, initial state, permitted tools, success criteria, evaluation cases, execution harness, and graders. Record the final outcome and useful trace evidence, including tool actions, cost, latency, and termination reason. Reset environments between trials to avoid leakage. Distinguish outcome correctness from process requirements such as whether the agent stayed within permissions.

049 Code-based vs model-based vs human graders?

Intermediate

Code graders are precise and repeatable for executable criteria such as schema validity or database state. Model graders scale to nuanced judgments but can be biased or inconsistent. Humans provide domain judgment but are slower and can disagree. Use the cheapest reliable grader for each criterion, calibrate subjective grading with labeled examples, and reserve expert review for ambiguous or consequential cases.

Advanced 1 Q&A

129 How do you estimate agent reliability from repeated trials?

Advanced

Define success and run representative cases repeatedly under controlled conditions. Report sample sizes, variability, and confidence intervals appropriate to the sampling design. Separate within-task randomness from differences between task types. Do not extrapolate a small set of easy successes into a broad reliability guarantee.

Grading and datasets

Basic 1 Q&A

130 **What is an evaluation rubric?**

Basic

A rubric states the criteria and evidence used to grade a result, such as factual support, task completion, and permission adherence. Define what counts as success, partial success, and failure before reviewing outputs. Concrete examples improve agreement. A vague instruction to judge quality produces scores that are difficult to interpret.

Intermediate 2 Q&A

050 **What is LLM-as-a-judge, and what are its pitfalls?**

Intermediate

An LLM judge evaluates another output using a rubric and provided evidence. Pitfalls include position bias, preference for verbosity, sensitivity to wording, correlated model errors, and prompt injection inside the evaluated content. Require rubric-specific judgments, randomize comparisons where useful, and calibrate against human labels. Treat judge scores as measurements with uncertainty, not objective proof of correctness.

051 **How do you build an eval set from scratch?**

Intermediate

Start with the user tasks and failure modes the product must handle. Collect realistic, permissioned examples, add edge and adversarial cases, and define expected outcomes or grading rubrics. Keep a held-out set separate from prompt tuning, label provenance, and prevent duplicates or leakage. Expand from production incidents and underperforming task slices. Version datasets so comparisons remain interpretable.

Advanced 1 Q&A

052 What is pass@k vs pass^k?

Advanced

Under a common definition, pass@k measures whether at least one of k attempts succeeds, while pass^k measures whether all k attempts succeed. If attempts are independent with identical success probability p, the probabilities are 1 − (1 − p)^k and p^k respectively. Independence is an assumption, not a given. Report the benchmark's exact definitions and sampling procedure; pass@k also needs a viable way to select a successful candidate in real use.

CATEGORY 09 / 34

Safety and security

2 topics · 7 questions

Prompt injection

Basic 1 Q&A

053 What is prompt injection?

Basic

Prompt injection is an attempt to make an AI system follow instructions from an attacker-controlled input that should not have authority over the task. A malicious passage might tell an agent to ignore its rules or disclose private data. Risk becomes more serious when the agent can access sensitive information or execute actions. The defense must include runtime boundaries as well as model guidance.

Intermediate 1 Q&A

054 Direct vs indirect prompt injection?

Intermediate

Direct injection is delivered through an attacker’s message or other direct task input. Indirect injection is embedded in material the agent encounters, such as a web page, retrieved document, email, or tool result. Both try to cross an instruction boundary. Preserve source provenance, treat external content as data, and authorize actions independently of instructions found in that content.

Advanced 1 Q&A

055 Why can't prompt injection be solved with prompting alone?

Advanced

Natural-language rules are interpreted by the same model that processes attacker-controlled text, and models can misclassify authority or intent. A prompt cannot revoke credentials or prevent an external write once a permissive runtime executes it. Reduce impact with least privilege, isolated execution, server-side authorization, controlled data flows, and action review. Prompting is one layer, not a complete guarantee.

Defense architecture

Basic 1 Q&A

131 What does least privilege mean for an agent?

Basic

Give the agent and its tools only the data access and operations required for the authorized task. Separate sensitive capabilities, scope credentials, and revoke access when it is no longer needed. This limits the impact of mistakes or injection. A broad administrator credential contradicts least privilege even if the prompt asks the agent to be careful.

Intermediate 1 Q&A

058 **What is sandboxing, and why do agents need it?**

Intermediate

Sandboxing constrains an execution environment's filesystem, network, process capabilities, credentials, and resource use. It limits the impact of buggy or malicious code an agent may run. A container alone is not automatically an adequate boundary for every threat model. Configure actual isolation, credential scope, time and memory limits, and controlled artifact transfer, and keep enforcement outside the model's control.

Advanced 2 Q&A

056 **How do you defend against prompt injection?**

Advanced

Limit what an agent can read and change, isolate untrusted execution, validate tool arguments, and enforce user and tenant authorization on every operation. Restrict outbound destinations and credential access, and require review for actions that need it. Label untrusted sources and avoid interpreting tool content as new policy. Test attacks and inspect real traces; no single detector or prompt should be treated as a complete defense.

057 **What is the "lethal trifecta"?**

Advanced

The phrase commonly describes the combination of access to private data, exposure to untrusted content, and a route for communicating externally. An injected instruction can try to use all three to exfiltrate information. It is a useful threat-modeling shorthand rather than a formal proof that every such system will fail. Break or tightly control these connections, especially outbound transfers of sensitive data.

CATEGORY 10 / 34

Production: cost, latency and scale

2 topics · 10 questions

Performance and resilience

Basic 1 Q&A

132 **Why can a multi-step agent be slow even with a fast model?**

Basic

Each step may wait for tool execution, network responses, queues, validation, and another model call. Serial dependencies add up. Trace the critical path before optimizing. Faster token generation will not solve a slow downstream API or an architecture with unnecessary sequential calls.

Intermediate 3 Q&A

059 **How do you reduce the cost of an agent?**

Intermediate

Measure cost per successful task, including retries and tools. Remove unnecessary calls, scope context and tools, reuse eligible cached prefixes, and route simpler tasks to cheaper models when evaluation supports it. Use deterministic code for straightforward computation and cache safe reusable results. Set budgets and compare quality by task slice so a lower bill does not hide a higher failure rate.

060 **How do you reduce agent latency?**

Intermediate

Trace the critical path and separate model time, queueing, tool execution, and network overhead. Reduce avoidable serial steps, retrieve only necessary data, choose suitable models, and parallelize independent reads. Stream progress for perceived responsiveness, but do not confuse it with faster completion. Evaluate time to first useful result and full completion, especially p95 latency under realistic load.

061 **How do you handle 429 and 5xx errors from LLM APIs?**

Intermediate

Respect Retry-After when present, apply capped exponential backoff with jitter for retryable errors, and enforce retry and deadline budgets. Use rate-aware queues and circuit breakers to avoid amplifying outages. A 429 may reflect quota exhaustion rather than a brief overload. Consider compatible fallbacks under an explicit policy. Never blindly replay consequential tools after a timeout; check their outcome or use idempotency controls.

Advanced 1 Q&A

133 **How do you optimize latency without increasing incorrect actions?**

Advanced

Identify avoidable waiting and parallelize independent operations while preserving authorization, consistency, and review points. Evaluate end-to-end correctness and tail latency under load. Speculative writes are especially risky because they can have external consequences. A latency improvement that raises mistaken actions may violate the product's actual requirements.

Scaling and durable work

Basic 1 Q&A

134 **Why persist agent state outside a running process?**

Basic

Processes can restart, time out, or move between workers. Persisting task state and completed operation references enables recovery without pretending the run starts fresh. Store data in an appropriate durable service and protect it. Persistence still needs safe replay and external side-effect handling.

Intermediate 1 Q&A

135 **How should a queued agent task respond to cancellation?**

Intermediate

Persist cancellation, stop dispatching new work, and signal active operations when their APIs support it. Record which actions already completed or remain uncertain. Clean up reserved resources safely and return a truthful final state. Cancellation cannot always undo an external action already accepted by another service.

Advanced 3 Q&A

062 **How do you scale an agent service to millions of users?**

Advanced

Design for peak concurrent runs and their tool and token demand, not registered-user count alone. Separate request handling from queued workers, persist state, enforce tenant quotas, and bound concurrency to provider and downstream capacity. Autoscale using queue depth and saturation metrics. Include fairness, cancellation, backpressure, tenant isolation, observability, and load tests with realistic multi-step tasks.

063 **What is durable execution for agents?**

Advanced

Durable execution persists workflow progress so a run can survive process restarts and wait for events without holding a live process indefinitely. A durable engine may record completed steps and resume or replay orchestration. Guarantees vary by implementation. External side effects still need idempotency keys, deduplication, or reconciliation because a crash can occur after an action succeeds but before its result is recorded.

064 **What are long-running agents, and what makes them hard?**

Advanced

These agents operate across extended periods, multiple sessions, or external events. Challenges include stale assumptions, changing permissions, context loss, partial failures, duplicated actions, accumulated cost, and cancellation. Persist explicit state, checkpoint results, revalidate authorization before consequential actions, and use deadlines and budget limits. Prefer event-driven waits to a model loop that continuously polls without useful work.

CATEGORY 11 / 34

Agentic system design

2 topics · 10 questions

Customer and developer systems

Basic 1 Q&A

136 **What is the first step in designing an agent for a real use case?**

Basic

Define the user's goal, supported tasks, required data, available actions, and observable completion criteria. Identify which decisions need model judgment and which should be deterministic. Establish the authorized scope before choosing a framework. A concrete task model makes architecture and evaluation choices meaningful.

Intermediate 1 Q&A

137 **How do you separate a support agent's reasoning from refund authority?**

Intermediate

Let the model gather context and propose an action, while the refund service checks identity, eligibility, amounts, and review requirements. Use stable operation identifiers and record the provider result. Explain denied or exceptional cases without inventing policy. The service remains authoritative even if the model is confident.

Advanced 3 Q&A

065 **Design a customer support agent that can issue refunds.**

Advanced

Authenticate the user, fetch only authorized order data, and use deterministic policy checks to calculate refund eligibility and limits. Let the agent explain the options, while a dedicated write service rechecks policy and permissions. Review high-risk or exceptional cases, and use an idempotent refund operation with an audit trail. Handle unknown payment outcomes through reconciliation. Evaluate policy correctness, fraud exposure, customer resolution, and mistaken refunds.

066 **Design a coding agent.**

Advanced

Use a scoped repository workspace with readable project instructions, a bounded task plan, and sandboxed tools for inspection, edits, and relevant tests. Keep secrets out of model context and limit network access according to the task. Show a reviewable diff and report verification evidence. Gate merges or deployments according to the user's authority. Evaluate actual task behavior and regression risk, not merely whether code compiles.

067 **Design a deep research agent.**

Advanced

Translate the request into research questions, collect evidence from suitable sources, track provenance, and identify gaps before synthesizing. Search iteratively under source, time, and cost budgets. Prefer primary evidence, compare contradictory sources, and attach citations to specific claims. Separate established findings from inference and uncertainty. Evaluate source quality, claim support, coverage, and whether the final answer addresses the original question.

Platform and communication systems

Basic 1 Q&A

138 **What should a system-design answer explain beyond a diagram?**

Basic

Explain assumptions, data flow, identity, state ownership, failure handling, verification, and operational tradeoffs. Connect each component to a requirement rather than listing fashionable tools. State what is outside scope. A useful diagram supports the reasoning but does not replace the behavior and invariants of the system.

Intermediate 1 Q&A

139 **How do you design graceful degradation for an agent platform?**

Intermediate

Identify which capabilities can fail independently and define a useful bounded fallback, such as read-only results or draft preparation. Preserve task state and explain the affected limitation. Avoid silently switching to incompatible permissions or data handling. Test fallback behavior under actual dependency errors, not just a happy-path configuration flag.

Advanced 3 Q&A

068 **Design a rate limiter and quota system for an LLM platform.**

Advanced

Authenticate the caller and enforce limits by tenant, model, and resource type such as requests, tokens, concurrency, and spend. Use a token-bucket or similar design for rate limits and a consistent ledger for quotas. Reserve estimated usage when admitting work, then reconcile actual usage including streaming or failures. Coordinate shared capacity across workers, return useful retry guidance, and test fairness, races, and outage behavior.

069 **Design an agent platform for 1 million users.**

Advanced

Separate the control plane for identity, policy, configuration, and billing from the execution plane for isolated runs and tools. Use durable state, queues, scoped credentials, tenant-specific quotas, tracing, and versioned prompts and tools. Size capacity from peak concurrent sessions and workload distribution. Add cancellation, recovery, data retention controls, and staged releases. Validate cost per successful task and load-test provider bottlenecks before promising scale.

070 **Design an AI email assistant that drafts and sends replies.**

Advanced

Connect to mail with the minimum required scopes and retrieve only relevant messages. Treat incoming email and attachments as untrusted content. Generate a draft grounded in the thread, and present recipients, subject, body, and attachments for review when required. Authorize sending in a separate service, prevent duplicate sends, and record the provider message ID. Evaluate wrong-recipient risk, factual accuracy, unwanted commitments, and successful delivery.

CATEGORY 12 / 34

Behavioral and project questions

2 topics · 8 questions

Explaining your work

Basic 1 Q&A

071 **Walk me through an agent you built. What would you do differently?**

Basic

Use a truthful project narrative: the user problem, why an agent was appropriate, your architecture and tools, the constraints, and how you evaluated it. Then describe an observed limitation and a concrete improvement. If you have not built a deployed agent, say so and discuss a prototype as a prototype. Prepare a diagram, one trace, and real outcome measurements; do not invent production experience.

Intermediate 2 Q&A

072 **Tell me about a time an agent failed in an unexpected way.**

Intermediate

Choose a real incident and describe the intended behavior, actual outcome, impact, and evidence used to find the cause. Distinguish a model error from a tool, state, or policy failure. Explain containment, the fix, and the regression case you added. An honest answer can include uncertainty or an unresolved limitation. Use a simulation only if you explicitly label it as such.

073 **Describe a cost vs quality tradeoff you made.**

Intermediate

Explain the baseline, the resource bottleneck, the alternatives, and the quality measure that guided your choice. Discuss a real change such as using a smaller model for simple classification while keeping a stronger model for difficult cases. Report actual before-and-after data when available and explain which task slices regressed. If proposing an experiment rather than reporting experience, make that distinction explicit.

Advanced 1 Q&A

140 **How should you discuss a failed agent project in a senior interview?**

Advanced

Explain the original hypothesis, evidence, constraints, and decisions you owned, including why the approach seemed reasonable. Describe impact, what changed your mind, and whether you repaired or stopped the project. Connect lessons to concrete engineering practice. Honest tradeoffs and limits show judgment more clearly than an invented rescue story.

Impact and learning

Basic 1 Q&A

076 **How do you keep up with the fast-moving AI field?**

Basic

Describe a sustainable practice: follow primary documentation and research relevant to your work, reproduce selected claims in small experiments, and record results. Evaluate updates against your own tasks before adopting them. Share what changed your engineering decisions and why. Avoid listing tools as a substitute for judgment; knowing when a new capability does not improve your system is useful expertise.

Intermediate 2 Q&A

074 **Tell me about a time you improved an agent's reliability.**

Intermediate

Identify a reproducible failure pattern, explain how you measured it, and describe the targeted change such as tool validation, retrieval fixes, or idempotency. Show how you separated the effect of that change from unrelated updates. Discuss regression testing and remaining failure modes. Structure the answer around evidence and your own contribution; use only outcomes you actually observed.

075 **How did you measure the impact of an AI feature?**

Intermediate

Tie the feature to a user outcome such as successful task completion or reduced resolution time, then measure quality, adoption, cost, latency, and negative outcomes alongside it. Use a baseline and, where feasible, a controlled rollout or experiment. Separate correlation from causal claims. Report uncertainty and important segments, because a better average can hide harm to particular users or tasks.

Advanced 1 Q&A

141 **How do you explain an AI feature's impact when causal evidence is weak?**

Advanced

Describe the measured association, baseline, sample, and likely confounders. Avoid claiming the feature caused the change without supporting design. Explain what experiment or additional measurement would strengthen the conclusion. Report useful operational evidence while remaining precise about what it can and cannot establish.

CATEGORY 13 / 34

Vibe coding, voice coding and builder practice

3 topics · 10 questions

From a request to a system

Basic 1 Q&A

077 **What changes when a vibe coder starts thinking like a builder?**

Basic

The request moves from a broad outcome to an explicit system: users, pages, data, actions, constraints, and a review plan. Instead of only asking for an e-commerce site, define categories, product details, cart state, checkout, confirmation, and failure paths. Build in reviewable blocks and verify the connections. A longer prompt is not inherently better; clarity and observable acceptance criteria matter.

Intermediate 1 Q&A

078 How should you break an e-commerce build into blocks?

Intermediate

Start with the user journey and a page map. Define shared product and cart data, create a reviewable homepage, then build product selection and a persistent cart. Add shipping, checkout, and confirmation with explicit error handling. Verify each block with its preceding and following steps. Keep demo payment controls restricted to test or admin use, and ensure they cannot create a real paid-order state accidentally.

Advanced 1 Q&A

142 How do you turn a broad vibe prompt into a testable system specification?

Advanced

Translate the goal into actors, entities, states, actions, constraints, and acceptance scenarios. Map the full user journey and failure paths, then prioritize reviewable increments. Keep assumptions visible and resolve blockers before dependent work. Verify implemented behavior against those scenarios rather than judging success from visual polish alone.

Voice as an interface

Basic 1 Q&A

079 What is voice coding?

Basic

Voice coding uses spoken instructions as an interface to coding tools, often through speech recognition and an AI assistant. It can make idea capture and iteration more natural, but spoken input can still be ambiguous or mistranscribed. State the goal, the specific change, constraints, and review point. Confirm critical names and values before actions that are difficult to undo.

Intermediate 1 Q&A

080 **What makes a useful voice coding prompt?**

Intermediate

Use a short sequence: context, next block, required behavior, connections, and a stopping point. For example: “We are building the T-shirt store. Add size selection to the product page, keep it in the cart, and prevent checkout without a size. Show me the change before continuing.” Review the transcript and the implementation; clear speech does not replace validation.

Advanced 1 Q&A

143 **How should a voice-driven coding workflow handle consequential commands?**

Advanced

Show or read back critical interpreted details, distinguish a requested draft from an authorized write, and apply the same execution policy used for typed input. Verify names, paths, amounts, and destinations when ambiguity matters. Keep a recoverable transcript and operation result. Voice recognition confidence does not authorize an action.

Blueprints and verification

Basic 1 Q&A

144 **What is the purpose of a page-flow blueprint?**

Basic

It shows how pages or states connect and what each one is meant to do. It helps reveal missing transitions, dead ends, and incomplete journeys before detailed implementation. Include important success and error paths. A blueprint describes the intended system; the implementation still needs behavioral verification.

Intermediate 2 Q&A

081 What belongs in an interactive system blueprint?

Intermediate

Include pages or states, their purpose, key content, inputs, outgoing actions, and links to preceding and following states. Show success, empty, error, and permission states where relevant. Use clear routing and clickable previews rather than tangled connectors. Keep the blueprint connected to implementation evidence: a navigable mockup demonstrates structure, while functional checks demonstrate that the underlying behavior works.

082 How do you know a generated website works beyond looking finished?

Intermediate

Run the real user journey with representative data and inspect both the visible result and stored state. Check validation, back navigation, empty states, refresh behavior, access control, and failures at external services. For checkout, distinguish test transactions from real payment confirmation. Treat screenshots as visual evidence and functional tests as behavior evidence; neither alone establishes the complete system.

Advanced 1 Q&A

145 How do you keep a blueprint aligned with implemented functionality?

Advanced

Associate blueprint nodes and transitions with routes, data contracts, and acceptance scenarios. Update them when requirements change and review discrepancies during verification. Use traces or tests to confirm key flows. Avoid presenting a clickable prototype as evidence that payments, permissions, or persistence actually work.

CATEGORY 14 / 34

Data governance and human oversight

3 topics · 11 questions

Identity and data lifecycle

Basic 1 Q&A

146 **Why should agent data have a defined retention period?**

Basic

Retention limits reduce unnecessary accumulation and clarify what users can expect about stored conversations, memory, and traces. Choose periods based on purpose and applicable obligations, and implement deletion across relevant stores. Explain backup behavior. Keeping data indefinitely just because it might be useful creates avoidable exposure.

Intermediate 2 Q&A

083 **How should an agent enforce user and tenant permissions?**

Intermediate

Bind every tool action and retrieval request to the authenticated user and tenant, then check authorization in the service that owns the data. Never let the model choose its own effective identity. Avoid retrieving unauthorized data and hoping the model will omit it. Recheck permission for delayed actions, use scoped credentials, and log decisions without exposing sensitive content unnecessarily.

084 **How should agent memory handle privacy and deletion?**

Intermediate

Store only information with a defined purpose and appropriate authorization, attach provenance and retention metadata, and restrict retrieval by user and tenant. Provide correction and deletion mechanisms across primary storage, indexes, and caches according to policy. Explain backup retention limitations. A memory summary is still user data and may retain sensitive facts even after the original conversation is removed.

Advanced 1 Q&A

147 **How do you verify deletion across an agent's derived data stores?**

Advanced

Map source records to chunks, embeddings, summaries, caches, and exported artifacts, then test removal or access blocking according to policy. Track completion and exceptions without retaining the deleted content unnecessarily. Include backup retention in documented behavior. Deleting one database row is not evidence that all derived copies are gone.

Human review and accountability

Basic 1 Q&A

085 **When should a human review an agent action?**

Basic

Use review when policy, user instructions, uncertainty, or consequence makes autonomous execution inappropriate. Present the exact action, affected parties, important inputs, and expected side effects so approval is informed. Avoid repeatedly requesting approval for already authorized routine work. Bind approval to the specific operation and re-review if material details change before execution.

Intermediate 1 Q&A

086 **What should an agent audit trail record?**

Intermediate

Record the user or service identity, task and run IDs, configuration versions, relevant evidence references, tool requests, authorization decisions, execution outcomes, timestamps, and final status. Correlate retries with stable operation IDs. Redact secrets and apply retention and access controls to traces. The goal is to reconstruct what happened and why without turning observability into an uncontrolled repository of private data.

Advanced 1 Q&A

148 **How do you ensure human oversight is meaningful rather than ceremonial?**

Advanced

Present the concrete action, relevant evidence, uncertainty, and consequences at a point where the reviewer can still change the outcome. Provide sufficient time and authority to reject or revise it. Measure reviewer workload and error detection. An approve button after an irreversible action is not effective oversight.

Product boundaries

Basic 1 Q&A

149 **What should an agent say when it cannot finish a task?**

Basic

State what was completed, what is blocked or unverified, and the specific next step needed. Link useful artifacts and distinguish a missing capability from a failed action. Do not imply success to make the interaction feel complete. A clear partial result is better than a fabricated outcome.

Intermediate 2 Q&A

087 **How do you choose an appropriate level of autonomy?**

Intermediate

Assess task ambiguity, potential impact, reversibility, available verification, and the user's authorization. Start with deterministic processing or draft-only assistance where that meets the need, then expand autonomy based on measured reliability. Keep high-consequence actions behind independent policy checks. Evaluate the full operating mode, including escalation burden, rather than only the model's isolated accuracy.

088 **How should an agent communicate uncertainty and failure?**

Intermediate

State what was completed, what remains unverified, the evidence available, and the next useful step. Separate a tool timeout from a confirmed failure: an unknown outcome may require checking state before retrying. Avoid fabricated success or unsupported confidence percentages. Give concise actionable explanations and preserve enough task state for safe continuation or human handoff.

Advanced 1 Q&A

150 **How do you evaluate whether autonomy should expand for a task?**

Advanced

Use representative trials and controlled deployment to measure verified success, unsafe actions, escalation burden, and user value at the current scope. Identify which additional decisions are supported by evidence and independent controls. Expand incrementally with rollback. Strong performance on low-consequence tasks does not justify unrelated high-consequence authority.

CATEGORY 15 / 34

Planning, reasoning and task state

3 topics · 10 questions

Plans and decomposition

Basic 1 Q&A

089 **What is the difference between a plan and a verified outcome?**

Basic

A plan describes intended work and dependencies; an outcome is an observed result supported by evidence. An agent may produce a convincing plan without completing any action. Track steps as pending, running, completed, blocked, or failed, and mark completion only after checking the appropriate tool result or artifact. Revise the plan when new evidence changes the task.

Intermediate 1 Q&A

090 **How detailed should an agent plan be?**

Intermediate

Make the plan detailed enough to expose dependencies, permissions, and verification points without freezing every low-level action in advance. Use shorter horizons when the environment is uncertain and refine after observations. A task list is useful only if it influences execution and gets updated. Avoid spending more resources maintaining an elaborate plan than the task warrants.

Advanced 1 Q&A

151 **How do you plan under uncertain dependencies?**

Advanced

Represent assumptions and dependencies explicitly, prioritize observations that resolve important uncertainty, and keep near-term steps executable. Use contingencies for likely failures and revise after tool results. Bound planning overhead. A detailed fixed sequence is fragile when the environment can invalidate its prerequisites.

Evidence and reasoning

Basic 1 Q&A

152 **What is an assumption in an agent task?**

Basic

An assumption is a proposition used to proceed without complete evidence, such as which environment or user role applies. State assumptions that materially affect the outcome and verify them when feasible. Do not silently turn a guess into a fact. Ask for clarification when a required decision cannot safely be inferred.

Intermediate 2 Q&A

091 Should you ask an agent to reveal its full chain of thought?

Intermediate

Do not rely on access to private internal reasoning. Ask for a concise decision summary, stated assumptions, evidence, and verifiable tool outputs instead. These are useful for review without treating a generated explanation as a faithful account of every internal computation. Trace what the system actually did and evaluate outcomes independently of how persuasive its explanation sounds.

092 How does an agent recover when its assumptions are wrong?

Intermediate

Detect disagreement between expected and observed results, preserve completed work, and identify which assumptions or dependencies are invalid. Gather targeted evidence before choosing a revised action. Use bounded retries and escalate when the task cannot proceed within scope. Record the change in task state so a later continuation does not repeat the failed approach.

Advanced 1 Q&A

153 How do you distinguish a persuasive explanation from execution evidence?

Advanced

An explanation describes a rationale; evidence records observed inputs, validated actions, and resulting state. Check source references, tool outcomes, and artifacts independently. A fluent explanation can rationalize an incorrect action after the fact. Evaluate actual task behavior and constraints rather than the apparent sophistication of the narrative.

State and coordination

Basic 1 Q&A

154 **Why should task state distinguish blocked from failed?**

Basic

Blocked means progress requires a missing dependency, permission, or decision; failed means an attempted step did not meet its contract. The distinction changes recovery and user communication. Record the cause and completed work. Treating both as a fresh task can duplicate actions or lose useful progress.

Intermediate 1 Q&A

093 **What belongs in an agent's explicit task state?**

Intermediate

Store the objective, accepted constraints, current status, completed and pending steps, artifact references, relevant evidence, budgets, and unresolved decisions. Keep credentials outside this state and avoid unnecessary private data. Explicit state should support resumption and audit; it is separate from a long transcript and should not depend on the model remembering an omitted conversation.

Advanced 1 Q&A

094 **How do you prevent duplicate or conflicting actions across workers?**

Advanced

Assign ownership of mutable resources and use atomic database updates, transactions, leases, or version checks where appropriate. Give external operations stable idempotency keys and deduplicate repeated work. Reconcile unknown outcomes before retrying. A shared natural-language plan is not a lock; correctness must be enforced by the state and execution layers under concurrent access and worker failure.

CATEGORY 16 / 34

Multimodal and computer-use agents

3 topics · 10 questions

Perception and interaction

Basic 1 Q&A

095 **What is a multimodal agent?**

Basic

A multimodal agent uses more than one information modality, such as text, images, audio, or video, while choosing actions toward a goal. A voice-driven coding assistant may combine speech recognition, text-based tool calls, and visual interface review. Each modality adds potential errors, so preserve useful source references and verify critical interpretations before acting on them.

Intermediate 1 Q&A

096 **How is computer use different from calling an API?**

Intermediate

Computer use interacts with a visible interface through clicks, typing, and other UI actions; API tools use explicit programmatic contracts. UIs can change, hide state, or introduce focus errors, while APIs often provide clearer identifiers and structured results. Prefer a suitable authorized API when available. When using a UI, inspect the current state and verify the result rather than assuming a click succeeded.

Advanced 1 Q&A

155 **How do you verify multimodal interpretations before consequential action?**

Advanced

Cross-check critical entities with structured data or additional observations, preserve the source reference, and surface unresolved ambiguity. Use task-specific validation for amounts, identities, and destinations. Evaluate the full perception-to-action chain. A visually plausible interpretation is insufficient when an error can trigger an external consequence.

Voice and visual ambiguity

Basic 1 Q&A

156 **Why should users review voice transcripts for critical details?**

Basic

Speech recognition can confuse names, numbers, technical terms, and negation. A brief confirmation of the interpreted detail can prevent the wrong operation. Prioritize details whose ambiguity changes the action rather than requiring review of every harmless sentence. The transcript is an aid to understanding, not proof of user authorization.

Intermediate 2 Q&A

097 **How do you reduce speech-recognition errors in an agent workflow?**

Intermediate

Use a transcript or read-back for critical entities such as recipients, filenames, amounts, and commands. Ask focused clarification when the ambiguity changes the action. Provide spelling or selection interfaces for unusual names, and let users correct the transcript before execution when consequences warrant it. Voice confidence scores alone do not establish that the intended action was understood.

098 **Can a screenshot prove that an agent completed a task?**

Intermediate

A screenshot can show a visible state at a particular time, but it may omit hidden errors, failed persistence, or downstream effects. Use it alongside structured evidence such as an order ID, saved file, database state, or verified message status. Distinguish the action the agent attempted from the outcome the relevant system actually recorded.

Advanced 1 Q&A

157 **How do you evaluate combined speech and coding failures?**

Advanced

Test realistic speech inputs with ambiguity, interruptions, corrections, and domain vocabulary, then check the resulting code and actions against the intended task. Separate recognition errors from interpretation and execution errors. Measure recovery and unwanted changes. A perfect text-prompt benchmark does not establish reliable voice-driven behavior.

Permissions and evaluation

Basic 1 Q&A

158 **Why does a browser agent need to verify the current page?**

Basic

The page can change after navigation, loading, or a previous action, and focus or selection may be wrong. Inspect the current state before targeting a control and check the result afterward. Verify account and destination where relevant. Reusing an old screen assumption can make a correct-looking click affect the wrong resource.

Intermediate 1 Q&A

099 **What boundaries should a browser or desktop agent have?**

Intermediate

Limit access to the accounts, sites, files, and actions authorized for the task. Treat page content as untrusted and avoid entering credentials or private data into unexpected destinations. Require the same policy checks for consequential UI actions as for API writes. Preserve user work, track current context, and verify destination and operation details before execution.

Advanced 1 Q&A

100 **How do you evaluate a computer-use agent?**

Advanced

Run realistic tasks in resettable environments and check resulting application state, not only click sequences. Include changed layouts, loading delays, dialogs, accessibility variations, incorrect focus, and interruption or cancellation. Track task success, unintended changes, time, cost, and intervention rate. A robust agent may use a different valid sequence than a reference trace, so grade outcomes and constraints separately.

CATEGORY 17 / 34

Model selection and adaptation

3 topics · 9 questions

Capability and routing

Basic 1 Q&A

159 **What capabilities matter when choosing a model for an agent?**

Basic

Match the model to the decisions it must make: instruction following, tool selection, structured output, domain knowledge, context handling, and any required image or audio input. Also check available hosting regions, latency, and limits. Evaluate complete tasks with realistic tools; a good conversational answer does not establish reliable action selection.

Intermediate 1 Q&A

160 **How do you design a model-routing policy?**

Intermediate

Define observable routing signals such as task type, input size, sensitivity, or failure history, then assign eligible models under explicit quality and cost constraints. Test the router and each destination together. Provide a fallback when classification is uncertain, and account for the extra latency and mistakes introduced by routing itself.

Advanced 1 Q&A

161 **How do you evaluate a model upgrade without confusing better language with better execution?**

Advanced

Replay representative tasks in resettable environments with comparable budgets, tools, and data. Grade actual final state, permission adherence, tool accuracy, cost, and tail latency. Analyze changed behavior by task slice and use a staged rollout. Preserve the previous model configuration for rollback where the provider supports it.

Fine-tuning and adaptation

Basic 1 Q&A

162 **What is fine-tuning, and when can it help an agent?**

Basic

Fine-tuning updates a model using training examples to improve behavior on a target distribution. It may help stable output conventions, tool selection, or specialized tasks when good examples exist. It does not automatically supply fresh knowledge or enforce authorization. Start by establishing whether the failure comes from the model rather than missing evidence or broken tools.

Intermediate 1 Q&A

163 **When should you use retrieval instead of fine-tuning?**

Intermediate

Use retrieval when answers depend on changing documents, user-specific data, or evidence that must be cited and access-controlled. Fine-tuning is more appropriate for repeatable behavior or task patterns, subject to evaluation. They can be combined, but neither repairs poor data quality. Compare approaches using the same held-out user tasks.

Advanced 1 Q&A

164 **How do you prevent training data from leaking into agent evaluation?**

Advanced

Keep evaluation cases and close duplicates out of training and example selection. Track dataset lineage, split related cases by source or time where useful, and check for near-duplicates. Use a sealed holdout plus fresh production-derived cases. Report that contamination cannot always be ruled out for externally trained base models.

Hosting and portability

Basic 1 Q&A

165 **What is the difference between hosted and self-hosted models?**

Basic

Hosted APIs delegate inference operations to a provider. Self-hosting gives an organization more control over deployment, model files, and infrastructure, while making it responsible for capacity, updates, isolation, and security. Neither option is automatically cheaper or private. Compare the complete operational and contractual requirements.

Intermediate 1 Q&A

166 **How do you make an agent portable across model providers?**

Intermediate

Keep application state and tools behind provider-neutral contracts, then isolate provider-specific message, tool, and streaming adapters. Record capability differences instead of pretending all models support the same features. Maintain contract tests and end-to-end evaluations for each supported model. Portability includes error and refusal handling, not just renaming request fields.

Advanced 1 Q&A

167 **How do you plan inference capacity for a self-hosted agent service?**

Advanced

Estimate concurrent requests, prompt lengths, output lengths, and multi-step amplification across typical and worst-case tasks. Load-test batching, queueing, accelerator memory, and token throughput under that distribution. Bound admission and expose backpressure. Include headroom, failure recovery, and model loading time rather than sizing solely from an average tokens-per-second benchmark.

CATEGORY 18 / 34

Knowledge ingestion and retrieval quality

3 topics · 9 questions

Ingestion and chunking

Basic 1 Q&A

168 **Why does document chunking matter for agents?**

Basic

Chunking determines which units can be retrieved and whether they contain enough context to support an answer. Chunks that are too small lose meaning; oversized chunks can dilute relevance and waste context. Preserve titles, source links, section relationships, and access metadata. Choose a strategy using actual retrieval tasks rather than one universal chunk size.

Intermediate 1 Q&A

169 **How should an agent system process tables and complex documents?**

Intermediate

Extract structure alongside text, retaining headers, units, page references, and relationships between cells. Store original artifacts for verification and flag uncertain extraction. Use specialized parsing or visual models when needed, then evaluate numerical and relational accuracy. Flattening a table into unordered text can make a plausible answer materially wrong.

Advanced 1 Q&A

170 **How do you keep an index synchronized with changing source documents?**

Advanced

Track stable source IDs, content versions, permissions, and deletion events. Make ingestion idempotent, update affected chunks and indexes, and expose freshness metadata. Reconcile missed events periodically. Test revocation and deletion propagation so removed content cannot continue appearing through stale chunks, caches, or secondary indexes.

Search and ranking

Basic 1 Q&A

171 **What is an embedding in a retrieval system?**

Basic

An embedding is a numerical representation used to compare items such as text queries and document passages. Similar vectors can indicate related meaning, depending on the embedding model and domain. A high similarity score is not proof that a passage is correct, current, or authorized. Retrieval still needs metadata checks and evidence inspection.

Intermediate 1 Q&A

172 **When does hybrid search help?**

Intermediate

Hybrid search combines lexical matching with semantic retrieval, helping when a query mixes exact identifiers and conceptual language. Merge or rerank candidates carefully because scores from different systems may not be comparable directly. Evaluate recall and precision on your domain. Exact product codes and rare technical terms often expose weaknesses in semantic-only retrieval.

Advanced 1 Q&A

173 **How do you diagnose whether a RAG failure is caused by retrieval or generation?**

Advanced

Check whether relevant authorized evidence exists in the corpus, was indexed correctly, entered the candidate set, survived ranking, and reached the final prompt. If adequate evidence was present but the answer was unsupported, investigate generation and instruction handling. Use stage-specific metrics and labeled cases rather than treating every wrong answer as a model hallucination.

Grounding and provenance

Basic 1 Q&A

174 **What makes a citation useful in an agent answer?**

Basic

A useful citation points to an accessible source that supports the nearby claim, ideally with a section, passage, or page reference. A real URL alone does not show support. Preserve the source and document version where possible. The reader should be able to distinguish direct evidence from the agent's interpretation.

Intermediate 1 Q&A

175 **How should an agent handle conflicting sources?**

Intermediate

Compare source authority, publication date, scope, and supporting evidence. Identify whether the apparent conflict is caused by different definitions or time periods. Explain unresolved disagreement instead of silently choosing a convenient statement. For consequential decisions, seek a primary source or escalate when the available evidence is insufficient.

Advanced 1 Q&A

176 **How do you evaluate citation faithfulness at scale?**

Advanced

Break answers into factual claims and check whether cited passages actually support each claim, including important qualifiers. Use automated checks for resolvability and calibrated model or human grading for semantic support. Sample by risk and task type. Evaluate unsupported claims separately from missing citations, because perfect citation formatting can still hide fabricated conclusions.

CATEGORY 19 / 34

Memory systems and personalization

3 topics · 9 questions

Memory design

Basic 1 Q&A

177 **What kinds of information should an agent remember across tasks?**

Basic

Persist useful, authorized information such as an explicit preference, a validated project convention, or an unresolved task. Keep transient guesses and sensitive material out unless there is a clear purpose and suitable controls. Attach provenance and scope. Memory should improve later decisions rather than become an indiscriminate archive.

Intermediate 1 Q&A

178 **How do episodic and semantic memory differ?**

Intermediate

Episodic memory records particular events or interactions; semantic memory records generalized facts or concepts. An agent might retain an incident trace as an episode and a verified service limit as a fact. Store the distinction explicitly so a single observation is not promoted into a universal rule without evidence.

Advanced 1 Q&A

179 **How do you design memory consolidation without reinforcing false beliefs?**

Advanced

Require provenance, confidence criteria grounded in evidence, and rules for resolving contradictory observations before promoting summaries into persistent facts. Keep original records available for review and support correction. Evaluate recall quality and harmful persistence over repeated sessions. Repeatedly summarizing the same unsupported claim does not increase its truth.

Recall and freshness

Basic 1 Q&A

180 **How does an agent retrieve a saved preference?**

Basic

The application identifies the authenticated user, searches that user's authorized memory, and inserts relevant preferences into the current context. The model does not automatically remember everything stored in a database. Use stable preference fields where possible, and distinguish an explicit user choice from an inferred tendency.

Intermediate 1 Q&A

181 **How do you prevent stale memory from overriding a current instruction?**

Intermediate

Give current explicit instructions appropriate precedence, attach timestamps and applicability conditions to stored memory, and revalidate facts that can change. Expire or revise obsolete preferences. When two authoritative instructions conflict and the action depends on the answer, ask for clarification rather than silently trusting an old summary.

Advanced 1 Q&A

182 **How do you evaluate long-term memory usefulness?**

Advanced

Run multi-session tasks that require remembering relevant facts while ignoring irrelevant or revoked ones. Measure retrieval precision, task benefit, correction propagation, and cross-user leakage. Include preference changes and contradictory evidence. Compare against a no-memory baseline so storage volume is not mistaken for improved user outcomes.

Privacy-aware personalization

Basic 1 Q&A

183 **What is the difference between a user profile and conversational history?**

Basic

A profile is a curated set of attributes or preferences intended for reuse. History is a chronological record that may contain temporary, irrelevant, or sensitive information. Deriving a profile is a separate processing decision. Make the purpose and controls clear, and avoid treating every statement in a conversation as a permanent user attribute.

Intermediate 1 Q&A

184 **How should users correct agent memory?**

Intermediate

Expose the remembered fact or preference and allow a correction or removal to update the authoritative store. Propagate the change to retrieval indexes and dependent summaries, with documented retention behavior for backups. Record provenance without preserving unnecessary sensitive content. Confirm what was changed rather than promising that every historical copy vanished instantly.

Advanced 1 Q&A

185 **How can shared organizational memory avoid cross-team data leakage?**

Advanced

Scope each item by identity, tenant, project, and applicable permissions, then enforce those checks before retrieval and at the data service. Recheck permissions after organizational changes. Separate broadly reusable knowledge from restricted source details and test membership transitions. A generated summary of confidential data remains confidential even if it omits the original document title.

CATEGORY 20 / 34

Workflow orchestration and event-driven agents

3 topics · 9 questions

Workflow structure

Basic 1 Q&A

186 **What is a state machine in an agent application?**

Basic

A state machine represents valid task states and allowed transitions between them. For example, a purchase may move from draft to reviewed to submitted, with separate failure states. It makes control flow inspectable and restricts invalid jumps. The model can recommend a transition while deterministic code enforces whether it is allowed.

Intermediate 1 Q&A

187 **How do you combine deterministic workflow steps with agent decisions?**

Intermediate

Keep predictable operations, validation, and authorization in code, and use the model for bounded decisions that require interpretation. Define the input, allowed outputs, and fallback for each model-driven step. Persist transitions and validate invariants. This makes an agent component replaceable without making the entire workflow unpredictable.

Advanced 1 Q&A

188 **How do you recover a workflow after its orchestration code changes?**

Advanced

Version workflow definitions and state schemas, then support compatible replay or an explicit migration for in-flight runs. Keep irreversible side effects recorded and avoid rerunning them during migration. Test representative old checkpoints against the new code. Some engines impose replay constraints, so use their actual guarantees rather than assuming arbitrary edits are safe.

Events and scheduling

Basic 1 Q&A

189 **When should an agent wait for an event instead of polling?**

Basic

Use an event-driven wait when progress depends on an external signal such as a completed payment, user response, or uploaded file. Persist the waiting state and resume when the event arrives. Poll only when events are unavailable or unreliable, with bounded intervals and deadlines. A waiting task does not require repeated model calls.

Intermediate 1 Q&A

190 **How do you handle duplicate or out-of-order events?**

Intermediate

Give events stable identifiers, deduplicate processing, and use version checks or event sequence information when the source provides it. Make transitions conditional on current state rather than arrival order alone. Reconcile against the source of truth when ordering is uncertain. Record ignored duplicates so operators can explain unexpected delivery patterns.

Advanced 1 Q&A

191 **How do you schedule recurring agent work safely?**

Advanced

Define the schedule, timezone, overlap policy, authorization scope, and a budget per run. Use stable run identifiers and prevent unintended concurrent executions. Handle daylight-saving changes, missed schedules, and retries according to explicit rules. Recheck credentials and user intent for ongoing external actions, and provide a clear cancellation mechanism.

Human checkpoints

Basic 1 Q&A

192 **What is a human-in-the-loop checkpoint?**

Basic

It is a defined pause where a person supplies a decision, correction, or authorization needed for the next step. Save enough state to resume without repeating prior work. Present the concrete artifact or action for review. A checkpoint is useful only if the system respects the decision and ties it to the correct operation.

Intermediate 1 Q&A

193 **How do you resume an agent after approval?**

Intermediate

Verify the approver's authority, match approval to the exact pending action and version, and confirm that material details have not changed. Record the decision and advance the persisted workflow. Revalidate live constraints before execution where needed. Avoid treating a generic yes from an unrelated conversation as authorization for a different action.

Advanced 1 Q&A

194 **How do you design approval expiry and cancellation races?**

Advanced

Represent approval, expiry, cancellation, and execution as competing state transitions governed by atomic checks. Permit execution only from a valid approved state, using the intended operation version. Define what cancellation means once an external system has accepted the action. Return an honest status such as already submitted rather than implying an in-flight side effect can always be undone.

CATEGORY 21 / 34

Distributed execution and reliability engineering

3 topics · 9 questions

Delivery semantics

Basic 1 Q&A

195 **What does at-least-once delivery mean for agent jobs?**

Basic

A job can be delivered more than once, especially after worker failures or lost acknowledgements. Consumers must tolerate repeated processing. Use stable job identities and idempotent side effects where possible. A queue's delivery guarantee does not automatically guarantee that an external action occurs only once.

Intermediate 1 Q&A

196 **What is an idempotency key and where should it be enforced?**

Intermediate

An idempotency key identifies one intended operation across retries. The service executing the write should atomically bind the key to the request and result, rejecting conflicting reuse. Keep keys stable for retries and distinct for genuinely new actions. Client-side memory alone cannot protect against simultaneous workers or a crash.

Advanced 1 Q&A

197 **Why is “exactly once” difficult across an agent and an external API?**

Advanced

The application cannot generally atomically commit its local state and an unrelated external side effect. A crash between the two can leave an unknown outcome. Use provider-supported idempotency, transactional outbox patterns, reconciliation, and operation status checks as appropriate. State the scope of any exactly-once claim rather than applying it to the entire distributed system.

Failure recovery

Basic 1 Q&A

198 **What should an agent do when a tool times out?**

Basic

Treat the result as unknown until the operation's semantics establish otherwise. A read may be safely retried, but a write may already have succeeded. Check operation status or retry using a stable idempotency key. Report uncertainty clearly instead of assuming a timeout means nothing happened.

Intermediate 1 Q&A

199 **How do circuit breakers protect agent services?**

Intermediate

A circuit breaker temporarily stops calls to a repeatedly failing dependency, reducing wasted work and allowing recovery. Track failures, open the breaker under a defined rule, and probe cautiously before closing it. Combine it with timeouts, backoff, and a useful degraded path. Do not classify user validation failures as dependency outages.

Advanced 1 Q&A

200 **How do compensation actions differ from transaction rollback?**

Advanced

Rollback can restore uncommitted changes within a transaction's guarantees. Compensation is a new action that attempts to offset an already completed external effect, such as issuing a refund after a charge. It may fail or leave an audit history and cannot always restore the original situation. Design and authorize compensations explicitly.

Concurrency and consistency

Basic 1 Q&A

201 **Why can parallel agents overwrite each other's work?**

Basic

Workers can read the same old state and produce competing updates without knowing another change occurred. Shared files and records need ownership or concurrency controls. Partition independent work when possible and merge deliberately. Natural-language instructions to be careful do not prevent a race at the storage layer.

Intermediate 1 Q&A

202 **When should you use optimistic concurrency for agent state?**

Intermediate

Use versioned updates when conflicts are possible but uncommon and retries are acceptable. Read a version, compute a change, and commit only if the version still matches; otherwise reload and reconsider. Avoid blindly replaying external actions on conflict. The retry must account for any work already completed.

Advanced 1 Q&A

203 **How do you maintain invariants across multiple workers and services?**

Advanced

Identify the service that owns each invariant, such as a remaining budget or order status, and enforce atomic state changes there. Use transactions for local invariants and explicit coordination or reconciliation for cross-service processes. Test partitions, retries, and stale reads. A global model conversation is not a substitute for a consistent accounting boundary.

CATEGORY 22 / 34

Agent testing and quality assurance

3 topics · 9 questions

Test layers

Basic 1 Q&A

204 **What is the difference between unit tests and agent evaluations?**

Basic

Unit tests check a narrow implementation contract under controlled inputs. Agent evaluations measure behavior across tasks where model choices may vary. Both are useful: a tool's validation can have deterministic tests while the full workflow has repeated outcome evaluations. Passing either layer alone does not establish end-to-end reliability.

Intermediate 1 Q&A

205 **What should a tool contract test cover?**

Intermediate

Check valid and invalid arguments, permission enforcement, result shape, known errors, timeouts, side effects, and idempotency where relevant. Include the behavior the agent depends on, such as whether a missing record is an empty result or an error. Keep tests focused on the external contract rather than duplicating internal implementation details.

Advanced 1 Q&A

206 **How do you test an agent that can change production-like data?**

Advanced

Use an isolated resettable environment with synthetic or appropriately de-identified data and scoped credentials. Record initial and final state, and verify both intended changes and forbidden changes. Replace irreversible external effects with faithful test endpoints where possible. Clearly distinguish simulation success from evidence about live-provider behavior.

Robustness and adversarial cases

Basic 1 Q&A

207 **Why should agent tests include missing or ambiguous information?**

Basic

Real requests omit fields, use conflicting terminology, or ask for unavailable capabilities. Test whether the agent gathers the right evidence, asks a focused question when necessary, and avoids inventing facts. Define which ambiguities it may reasonably resolve and which block execution. Otherwise a test set rewards confident guesses that fail in use.

Intermediate 1 Q&A

208 **What is a metamorphic test for an agent?**

Intermediate

It checks an expected relationship between outputs after a controlled input change. For example, reordering irrelevant documents should not change a supported conclusion, while changing an authorized spending limit should change eligible actions. Use domain-valid transformations and explicit expectations. Such tests reveal brittleness without requiring one exact text answer.

Advanced 1 Q&A

209 **How do you build a prompt-injection regression suite?**

Advanced

Include attacks through direct input, retrieved content, tool descriptions, and external documents. Vary presentation while testing concrete protected outcomes such as secret access, unauthorized writes, or destination changes. Reset the environment and inspect actual tool actions. Maintain benign lookalike cases so a defense is not rewarded simply for refusing every task.

Release confidence

Basic 1 Q&A

210 **Why should evaluation tasks be versioned?**

Basic

Versioning preserves the exact cases, environments, and grading rules behind a reported result. Otherwise an apparent improvement may come from an easier dataset or changed labels. Keep old results interpretable and document corrections. Version prompts, tools, and model configuration alongside the evaluation set.

Intermediate 1 Q&A

211 **How do you choose regression cases after an incident?**

Intermediate

Capture the smallest realistic case that reproduces the failure, then add nearby cases that distinguish the root cause from unrelated behavior. Include the protected outcome and expected recovery. Remove secrets and preserve necessary context. A test that checks only the incident's exact wording may miss the same failure in a slightly different form.

Advanced 1 Q&A

212 **How should you make a release decision when eval results conflict?**

Advanced

Prioritize explicit product requirements and risk thresholds, then inspect task slices, sample sizes, and grading reliability. A quality gain may not justify a regression in unauthorized actions or critical users. Seek targeted additional evidence, document accepted tradeoffs, and stage the rollout. Avoid collapsing every dimension into one opaque average score.

CATEGORY 23 / 34

Observability and incident operations

3 topics · 9 questions

Traces and diagnostics

Basic 1 Q&A

213 **What is an agent trace?**

Basic

A trace records the steps of a run, such as model calls, tool requests, timing, state transitions, and outcomes. It connects user-visible behavior to execution evidence. Include configuration versions and correlation IDs, while minimizing sensitive payloads. A transcript alone often lacks enough information to diagnose distributed failures.

Intermediate 1 Q&A

214 **Which trace fields are useful for debugging tool failures?**

Intermediate

Capture the run and operation IDs, tool version, validated argument metadata, authorization result, start and end times, status, retry count, and provider request ID where available. Redact secrets and sensitive contents. Distinguish a transport timeout from a confirmed business rejection so recovery can choose a safe next step.

Advanced 1 Q&A

215 **How can observability itself create security risk?**

Advanced

Traces can collect credentials, private documents, user messages, and generated sensitive inferences. Centralizing them broadens exposure and retention. Use redaction, access control, retention limits, and audited retrieval; avoid logging entire payloads by default. Include tracing vendors and exports in the data-governance model rather than treating logs as harmless metadata.

Service health

Basic 1 Q&A

216 **What metrics should an agent dashboard show?**

Basic

Track successful task completion, failures, escalation, latency, cost, tool errors, and cancellation. Separate user-request volume from model-call volume because each task may have many steps. Show important slices such as model, tenant tier, and task type without exposing private data. Metrics should support decisions, not just count generated tokens.

Intermediate 1 Q&A

217 **What is a useful service-level objective for an agent?**

Intermediate

An SLO defines a measurable target for a user-relevant outcome over a period, such as a specified share of supported tasks completing within a latency and correctness threshold. Define the eligible population and measurement process carefully. Pair availability with quality and safety indicators because a responsive agent can still produce unusable results.

Advanced 1 Q&A

218 **How do you detect a gradual quality regression in production?**

Advanced

Monitor stable task slices, sampled human or calibrated automated reviews, tool-error distributions, and outcome proxies with known limitations. Compare against release versions and control groups where feasible. Watch for changes in user mix and source data. A decrease in complaints alone may reflect lower usage rather than improved quality.

Incident response

Basic 1 Q&A

219 **What should an operator do first during an agent incident?**

Basic

Contain the affected capability, preserve relevant evidence, assess user impact, and identify whether external side effects continue. Use scoped controls such as disabling a tool or reverting a configuration rather than deleting evidence. Communicate confirmed facts and unknowns. Restoration should include checking task state, not only restarting workers.

Intermediate 1 Q&A

220 **How do you investigate an unintended external action?**

Intermediate

Correlate the user request, authorization policy, model proposal, actual executed arguments, and downstream result. Determine whether the cause was ambiguous intent, permission failure, injection, retry duplication, or a tool defect. Reconcile affected state and contact appropriate owners through authorized channels. Build the fix around the execution boundary that failed.

Advanced 1 Q&A

221 **What makes an agent postmortem actionable?**

Advanced

Document impact, timeline, contributing technical and organizational factors, containment, and the evidence supporting the root-cause analysis. Assign concrete improvements with owners and verification criteria. Include why existing tests or controls did not detect the issue. Avoid attributing everything to the model when tool design, permissions, or recovery behavior amplified the failure.

CATEGORY 24 / 34

Identity, authorization and secrets

3 topics · 9 questions

Identity boundaries

Basic 1 Q&A

222 **How are authentication and authorization different?**

Basic

Authentication establishes who is making a request; authorization determines what that identity may do to a resource. An agent may act on behalf of a user, but its tool calls still require resource-level checks. A logged-in session alone is not permission to read every record or perform every action.

Intermediate 1 Q&A

223 **What is a confused-deputy problem in an agent system?**

Intermediate

A more privileged component may be induced to use its authority on behalf of a less privileged requester. An agent with broad service credentials can become that deputy if tool requests are not bound to the real user's permissions. Enforce the caller's scope at the owning service and avoid allowing prompt text to select a more privileged identity.

Advanced 1 Q&A

224 **How do you design delegated authorization for long-running agents?**

Advanced

Bind each run to a verified principal and explicit delegated scope, with expiry, revocation, and auditability. Recheck authority before consequential operations, especially after long waits or membership changes. Use short-lived credentials or brokered access where appropriate. A permission granted when a run started may no longer be valid when it resumes.

Credential handling

Basic 1 Q&A

225 **Why should secrets stay out of agent prompts?**

Basic

Prompts can be logged, forwarded, included in traces, or exposed through injection and accidental output. Give tools access to credentials through a controlled runtime or secret broker instead. The model typically needs to know a capability exists, not the underlying token. Redaction reduces exposure but does not replace least privilege.

Intermediate 1 Q&A

226 **How do you scope credentials for individual tools?**

Intermediate

Grant only required operations and resources, separate reads from writes where practical, and bind access to the correct tenant and user context. Use distinct credentials or scoped tokens rather than a global administrator key. Rotate and revoke them through managed mechanisms. Validate that the downstream service actually enforces the requested scope.

Advanced 1 Q&A

227 **What should happen when a credential is revoked during a run?**

Advanced

Stop new operations that depend on that credential, record the blocked state, and avoid retries that silently substitute a more privileged identity. Determine whether already submitted external actions completed. Resume only with valid authorization according to the task's rules. Preserve enough state for a clear user explanation and safe continuation.

Access-policy enforcement

Basic 1 Q&A

228 **Where should an agent's permissions be enforced?**

Basic

Enforce permissions in the runtime and the service that owns the resource, before the operation occurs. Prompts can explain policy but cannot act as the only gate. Validate both operation type and resource scope. A tool that accepts arbitrary tenant IDs must not trust the model to choose only authorized ones.

Intermediate 1 Q&A

229 **What is the difference between RBAC and ABAC for agent tools?**

Intermediate

Role-based access control grants permissions through roles. Attribute-based access control uses attributes such as user, resource, tenant, time, or workflow state. Systems often combine them. Choose a policy that can express the required constraints and explain decisions. The model can supply task context, but trusted services must determine authoritative attributes.

Advanced 1 Q&A

230 **How do you test that an agent cannot cross tenant boundaries?**

Advanced

Create adversarial cases with similar record names, guessed IDs, shared indexes, cached results, and changing memberships. Check retrieval, tools, exports, traces, and memory, not just the UI. Verify denial occurs before sensitive data reaches the model. Use automated policy tests plus end-to-end runs under realistic identities.

CATEGORY 25 / 34

Agent skills, instructions and capability discovery

3 topics · 9 questions

Instruction packages

Basic 1 Q&A

231 **What is an agent skill or instruction package?**

Basic

It is a reusable set of task-specific guidance, examples, scripts, or references that an agent can load when relevant. Formats differ across products. A skill can improve consistency but does not inherently grant permission or establish trust. Treat any included executable content and external actions according to the host's security model.

Intermediate 1 Q&A

232 **How should an agent decide which skill to load?**

Intermediate

Match the skill's declared purpose and prerequisites to the actual task, rather than using a keyword alone. Prefer the narrowest relevant instructions and avoid loading unrelated material into context. Check version and source. If a required skill is unavailable, report the specific gap and continue independent authorized work where possible.

Advanced 1 Q&A

233 **How do you defend against malicious or outdated skill packages?**

Advanced

Use trusted distribution, review executable content and external destinations, pin versions where appropriate, and audit updates. Separate descriptive instructions from actual permission grants and enforce access outside the package. Test important workflows after changes. A familiar skill name does not prove that its contents or dependencies remain safe.

Capability discovery

Basic 1 Q&A

234 **Why is tool discovery useful for large agent systems?**

Basic

It lets the agent obtain relevant tool descriptions without placing every integration in every request. This can reduce context and selection ambiguity. Discovery must be scoped by the user's authorized capabilities. Finding a tool establishes availability, not authorization for every operation it exposes.

Intermediate 1 Q&A

235 **How do you evaluate a dynamic tool-discovery system?**

Intermediate

Measure whether the necessary tools are retrieved, whether irrelevant or forbidden tools appear, and whether the agent chooses and calls the right contract. Include rare and ambiguous requests plus tools with similar names. Compare total task success, context cost, and latency against a fixed tool set. Discovery errors can masquerade as reasoning failures.

Advanced 1 Q&A

236 **How do you manage tool-schema changes without breaking active agents?**

Advanced

Version contracts, preserve compatibility where feasible, and bind active runs to a supported version or migrate explicitly. Validate arguments and expose clear deprecation errors. Update discovery indexes and prompt examples together. Test old checkpoints against changed schemas so a resumed run cannot accidentally reinterpret a parameter or side effect.

Instruction conflicts

Basic 1 Q&A

237 **How should an agent treat instructions found inside retrieved documents?**

Basic

Treat them as document content unless a trusted mechanism explicitly establishes their authority for the task. A document may describe commands without authorizing execution. Preserve provenance and continue following the actual user request and governing policies. This distinction is central when reading code, emails, web pages, or shared files.

Intermediate 1 Q&A

238 **How do you handle conflicting project instructions?**

Intermediate

Identify the applicable scope and established precedence, compare the conflicting requirements, and use current task authorization where it resolves the issue. If a consequential decision remains ambiguous, ask a focused question while continuing unaffected work. Record the resolution. Do not silently choose whichever instruction makes execution easiest.

Advanced 1 Q&A

239 **How do you evaluate instruction-following across nested contexts?**

Advanced

Construct tasks with trusted instructions, scoped project guidance, and untrusted documents that contain conflicting directives. Grade both the completed task and whether protected boundaries were respected. Vary wording and document placement to prevent rote matching. Inspect tool execution because a compliant final message can conceal an unauthorized action earlier in the run.

CATEGORY 26 / 34

Learning, feedback and optimization

3 topics · 9 questions

Feedback signals

Basic 1 Q&A

240 **What kinds of feedback can improve an agent system?**

Basic

Useful signals include verified task outcomes, user corrections, tool errors, expert labels, and well-designed preference comparisons. Each has bias: user satisfaction may reward confidence, and tool success may not mean the user's goal was met. Record provenance and evaluate before using feedback to change behavior.

Intermediate 1 Q&A

241 **How do you turn production failures into useful learning data?**

Intermediate

Identify the root cause, reconstruct the necessary context, remove sensitive data, and label the correct action or outcome. Include nearby counterexamples so a fix generalizes. Separate tool defects and authorization failures from model behavior that training could change. Keep a held-out evaluation set that is not recycled into tuning.

Advanced 1 Q&A

242 **How do you prevent feedback loops from reinforcing bad behavior?**

Advanced

Audit who supplies feedback, what it rewards, and whether the signal correlates with real task success. Track distribution shifts and adversarial manipulation. Require independent evaluation before promoting changes, especially from self-generated labels. A system that judges and trains only on its own confident outputs can amplify errors.

Reinforcement learning concepts

Basic 1 Q&A

243 **What is reinforcement learning in the context of agents?**

Basic

Reinforcement learning optimizes a policy using feedback about actions and outcomes in an environment. An LLM-based agent may be trained with task rewards or preference-derived signals, but many deployed agents do not learn online at all. Distinguish training the model from changing prompts, memory, or routing during application operation.

Intermediate 1 Q&A

244 **What is reward hacking?**

Intermediate

Reward hacking occurs when a policy improves the measured reward without meeting the intended objective. For example, an agent might close support tickets quickly while leaving customers unresolved. Use outcome checks, adversarial cases, and multiple measures that expose shortcuts. A more detailed reward is not automatically safe if it still omits important behavior.

Advanced 1 Q&A

245 **How would you evaluate a policy optimized for multi-step tool use?**

Advanced

Use environments with independently checked outcomes, explicit action constraints, and held-out tasks that differ from training. Measure total success, unsafe actions, cost, and recovery, not just per-call accuracy. Test whether reward-improving shortcuts exploit simulator artifacts. Assess generalization to realistic tool failures before allowing broader deployment.

Continuous improvement

Basic 1 Q&A

246 **Can an agent improve without changing model weights?**

Basic

Yes. Better tool contracts, retrieval, instructions, memory selection, validation, and workflow design can improve task success. These changes alter the environment and information available to the model. Measure them with controlled comparisons. Calling the improvement learning should not obscure whether the model itself was actually updated.

Intermediate 1 Q&A

247 **How do you prioritize agent improvements?**

Intermediate

Group failures by frequency, impact, and root cause, then estimate the expected benefit and effort of each change. Fix deterministic defects and broken permissions before adding more reasoning loops. Use a representative baseline and a small experiment to validate the intervention. Track cost per successful task as well as accuracy.

Advanced 1 Q&A

248 **How do you govern automatic prompt or policy optimization?**

Advanced

Limit the search space, use approved training data, and evaluate candidates against held-out quality and policy criteria. Version every candidate and require appropriate review before production promotion. Keep rollback available and prevent optimizers from modifying their own evaluation controls. Improvement in one metric is insufficient evidence for unrestricted self-modification.

CATEGORY 27 / 34

Coding agents and software engineering

3 topics · 9 questions

Repository understanding

Basic 1 Q&A

249 **What should a coding agent inspect before changing a repository?**

Basic

Read applicable project instructions, locate the relevant implementation, and understand the current behavior and local conventions. Inspect the working tree so existing user changes are preserved. Identify how the change can be verified. Avoid reading unrelated secrets or assuming the repository is clean.

Intermediate 1 Q&A

250 **How do you give a coding agent a useful task specification?**

Intermediate

State the observed problem, expected behavior, scope, constraints, and acceptance evidence. Include a concrete failing scenario or reproduction when available. Let the agent resolve routine implementation details within that scope. A specification should explain the user outcome rather than prescribe every edit without understanding dependencies.

Advanced 1 Q&A

251 **How should a coding agent manage changes in a shared workspace?**

Advanced

Inspect current changes, give one owner to each overlapping file or region, and coordinate integration explicitly. Avoid resetting or overwriting work the agent did not create. Use isolated branches or worktrees when appropriate and verify the integrated result. Independent tasks are easier to parallelize than multiple agents rewriting the same core module.

Implementation and verification

Basic 1 Q&A

252 **Why is a compiling program not enough evidence of a successful agent change?**

Basic

Compilation checks syntax and some type constraints, but the feature can still violate requirements, break a user flow, or mishandle data. Verify behavior appropriate to the change and inspect the diff. For visible interfaces, review the rendered result. Report any verification limits clearly.

Intermediate 1 Q&A

253 **How should a coding agent choose which tests to run?**

Intermediate

Select checks that cover the changed behavior and likely regressions, following applicable project requirements. Prefer meaningful existing tests and focused manual or integration checks when appropriate. Broaden testing if failures or unresolved risk justify it. Avoid writing trivial tests that simply repeat the implementation without checking a useful contract.

Advanced 1 Q&A

254 **How do you evaluate a coding agent on real maintenance work?**

Advanced

Use representative tasks with hidden acceptance checks, realistic repository state, and constraints on unrelated changes. Grade functional behavior, regression risk, maintainability, and whether the agent preserved user work. Track tool use, time, and cost. Prevent evaluation leakage from issue answers or reference patches where possible.

Delivery and review

Basic 1 Q&A

255 **What should an agent include in its change report?**

Basic

Explain what changed, why it matters, how it was verified, and any material limitations. Link the relevant artifacts or files and avoid claiming tests ran when they did not. The report should let a reviewer assess the result without reading the entire interaction history.

Intermediate 1 Q&A

256 **How should a coding agent prepare a reviewable pull request?**

Intermediate

Keep the final scope coherent, inspect the diff, complete relevant checks, and write a description around the concrete problem and resulting behavior. Include validation and meaningful risks. Follow the repository's contribution process and task authorization. A draft pull request can make work reviewable without implying production deployment approval.

Advanced 1 Q&A

257 **How do you prevent a coding agent from treating deployment as a harmless final step?**

Advanced

Model deployment, migrations, secret changes, and destructive operations as explicit capabilities governed by authorization and environment policy. Separate preparation from execution and present concrete artifacts for review when approval is required. Enforce controls in deployment tooling. A successful local test does not authorize a production change.

CATEGORY 28 / 34

Research, analysis and knowledge-work agents

3 topics · 9 questions

Research design

Basic 1 Q&A

258 **How should a research agent scope a broad question?**

Basic

Identify the decision or learning goal, key subquestions, relevant time period, and constraints on sources or depth. Start with a manageable evidence plan and adjust it when sources reveal gaps. If scope remains ambiguous, state assumptions or ask a focused question. More searches are not automatically better research.

Intermediate 1 Q&A

259 **How do you avoid citation laundering in agent research?**

Intermediate

Check the underlying primary evidence rather than citing one article that repeats another unsupported claim. Track whether a source reports firsthand findings or summarizes someone else. Attach citations to the claims they actually support. If the original source is unavailable, explain that limitation instead of presenting secondary reporting as direct verification.

Advanced 1 Q&A

260 **How do you evaluate a research agent on an open-ended task?**

Advanced

Define a rubric for coverage, source authority, factual support, uncertainty handling, and usefulness for the user's decision. Use expert review and calibrated automated checks where appropriate. Judge the evidence trail and final synthesis rather than the number of searches. Include tasks with contradictory sources and questions that cannot be resolved from available evidence.

Data analysis

Basic 1 Q&A

261 **When should an agent use code for analysis?**

Basic

Use code for reproducible calculations, data transformations, statistical analysis, and plots when the task benefits from exact execution. Inspect the data schema and units first. Validate results against simple checks and explain assumptions. A language-model estimate should not replace a calculation that can be executed reliably.

Intermediate 1 Q&A

262 **How do you prevent a data-analysis agent from making unsupported causal claims?**

Intermediate

Distinguish descriptive patterns, predictive associations, and causal effects. Check study design, confounders, selection bias, and whether an intervention or identification strategy supports causality. Present uncertainty and alternative explanations. Running a regression alone does not establish that changing one variable will cause the predicted outcome.

Advanced 1 Q&A

263 **How do you build a reproducible analysis workflow for an agent?**

Advanced

Version input references, code, dependencies, random seeds where relevant, and analysis choices. Save intermediate artifacts and a clear execution path. Use isolated compute with scoped data access and record validation checks. Reproducibility should include the interpretation and assumptions, not only rerunning a notebook to produce the same chart.

Writing and synthesis

Basic 1 Q&A

264 **How should a writing agent preserve factual accuracy?**

Basic

Separate source-supported facts from stylistic choices and original inference. Keep evidence references for factual claims and flag missing information rather than inventing it. Review names, numbers, quotations, and dates carefully. A polished sentence can still be factually wrong.

Intermediate 1 Q&A

265 **How do you coordinate research and writing agents?**

Intermediate

Define an evidence handoff containing claims, sources, uncertainty, and unresolved questions before drafting. Give the writer authority to improve presentation without silently changing factual meaning. Review the final text against the evidence. Separate roles help only when their contracts prevent unsupported additions during synthesis.

Advanced 1 Q&A

266 **How do you evaluate an agent-generated briefing for decision makers?**

Advanced

Check whether it answers the actual decision question, presents relevant options and tradeoffs, supports claims, and distinguishes uncertainty. Assess omissions that could change the decision, not just factual errors in included text. Use domain reviewers for consequential topics. Concision is valuable when it preserves the information needed to act responsibly.

CATEGORY 29 / 34

Business operations and transactional agents

3 topics · 9 questions

Customer support and CRM

Basic 1 Q&A

267 **What tasks are suitable for a customer-support agent?**

Basic

Agents can classify requests, retrieve authorized account information, draft responses, and guide users through supported processes. Higher-consequence changes need independent policy and permission checks. Define escalation conditions and verify resolution. A fast response is not the same as a solved customer problem.

Intermediate 1 Q&A

268 **How do you stop an agent from inventing company policy?**

Intermediate

Retrieve the applicable current policy from an authorized source and require explanations to stay within it. Use deterministic checks for rules that control transactions. If a case falls outside policy, escalate rather than improvise commitments. Track policy versions so a response can be audited later.

Advanced 1 Q&A

269 **How do you evaluate a support agent beyond ticket deflection?**

Advanced

Measure verified resolution, repeat contacts, escalation appropriateness, policy adherence, and customer impact alongside cost and response time. Audit difficult and sensitive cases separately. Ticket closure can be gamed or misinterpreted. Use a baseline and controlled rollout where feasible to estimate the feature's real effect.

Sales and scheduling

Basic 1 Q&A

270 **How can an agent help with scheduling without creating confusion?**

Basic

Collect constraints, inspect authorized calendar availability, propose clear options with timezones, and distinguish tentative holds from confirmed events. Verify participants and meeting details before writes. Record the event result. Treat cancellations and rescheduling as separate actions with their own consequences.

Intermediate 1 Q&A

271 **What controls matter for an outbound sales agent?**

Intermediate

Respect authorized audience, consent and applicable communication rules, approved claims, frequency limits, and opt-outs. Keep generated personalization grounded in permitted data. Review or constrain sending according to policy and user authorization. Draft quality does not justify unrestricted messaging at scale.

Advanced 1 Q&A

272 **How do you prevent repeated outreach across multiple agent workflows?**

Advanced

Use a shared contact and campaign ledger with stable identifiers, atomic send eligibility checks, and provider message reconciliation. Deduplicate retries and coordinate frequency limits across teams. Recheck opt-out state at send time. A per-agent local memory cannot enforce organization-wide contact policies.

Payments and orders

Basic 1 Q&A

273 **Why should payment confirmation come from a payment system?**

Basic

A UI success screen or model statement can be wrong or fabricated. Confirm payment using authenticated provider events or status checks and the matching order, amount, and currency. Handle pending or unknown states explicitly. Do not treat a demo button as evidence of a real charge.

Intermediate 1 Q&A

274 **How should an agent manage a partially completed order?**

Intermediate

Persist which steps succeeded and which remain pending, then reconcile external states before retrying. Inform the user of the actual status and available next actions. Use idempotent operations for payment and submission. Avoid starting a second order merely because the first run lost its response.

Advanced 1 Q&A

275 **How do you design an agent that can purchase within a budget?**

Advanced

Enforce identity, allowed vendors, item constraints, spend limits, and required approvals outside the model. Reserve budget atomically before submission and reconcile actual charges. Present material purchase details for review when required. Include taxes, shipping, duplicate retries, and cancellations in the accounting model.

CATEGORY 30 / 34

Agent platforms and developer experience

3 topics · 9 questions

Platform building blocks

Basic 1 Q&A

276 **What core services does an agent platform need?**

Basic

Typical building blocks include identity, run orchestration, model access, tool execution, persistent state, evaluation, observability, and configuration management. Add only what the product needs. Define boundaries so applications do not each recreate security and recovery logic inconsistently.

Intermediate 1 Q&A

277 **How do you design a reusable tool gateway?**

Intermediate

Authenticate callers, scope available tools, validate contracts, enforce authorization, and correlate execution with run IDs. Provide consistent error semantics, deadlines, and audit events. Keep domain business rules in the owning service. A gateway should not become a universal administrator bypass simply because every agent uses it.

Advanced 1 Q&A

278 **How do you manage backwards compatibility for a shared agent platform?**

Advanced

Version APIs, state formats, tool contracts, and supported model adapters. Define deprecation and migration policies for applications and in-flight runs. Use contract tests and staged releases with representative clients. Platform changes can affect many agents at once, so blast radius and rollback deserve explicit planning.

Developer workflows

Basic 1 Q&A

279 **What should a local agent development environment provide?**

Basic

Provide safe test credentials or mocks, inspectable traces, representative data, configurable models, and a way to reset state. Make consequential external actions clearly distinct from simulation. Keep setup reproducible and document required permissions. A local demo should not accidentally operate on production resources.

Intermediate 1 Q&A

280 **How do you make agent debugging practical for application teams?**

Intermediate

Expose correlated traces, deterministic tool replays where possible, versioned configurations, and compact failure summaries. Let developers inspect relevant state without revealing unnecessary sensitive content. Provide fixtures for common failures and a way to reproduce a run in isolation. A single final model response rarely explains a multi-step defect.

Advanced 1 Q&A

281 **How should an agent platform support cost attribution?**

Advanced

Associate model calls, tools, storage, and execution resources with run, tenant, application, and configuration identifiers. Reconcile estimated with actual billed usage and account for retries and background work. Expose cost per successful task as well as raw spend. Avoid double-counting shared infrastructure allocations as if they were direct provider charges.

Isolation and deployment

Basic 1 Q&A

282 **Why separate the control plane from the execution plane?**

Basic

The control plane manages configuration, identity, and policy; the execution plane performs runs and tool work. Separation can improve security, scaling, and operational clarity. Define trusted communication between them and avoid giving workers unrestricted configuration authority. The exact split should match the platform's complexity.

Intermediate 1 Q&A

283 **How do you isolate tenants in an agent execution service?**

Intermediate

Scope credentials, data, storage, queues, and resource limits to the correct tenant. Use appropriate process or compute isolation and enforce access at service boundaries. Include logs and caches in the isolation model. A tenant label in a prompt is not a technical boundary.

Advanced 1 Q&A

284 **How do you design regional or data-residency controls for agents?**

Advanced

Map where model inference, tools, retrieval, memory, traces, backups, and support access process data. Route only to services that meet the applicable contractual and legal requirements. Enforce region policy in infrastructure and service configuration. Keeping the database in one region is insufficient if prompts or traces are exported elsewhere.

CATEGORY 31 / 34

Edge, robotics and embodied agents

3 topics · 9 questions

Physical-world interaction

Basic 1 Q&A

285 **What makes an embodied agent different from a text-only agent?**

Basic

An embodied agent acts through a physical system such as a robot and receives observations from sensors. Actions can have immediate real-world consequences, and perception is noisy and incomplete. Use explicit control and safety layers appropriate to the hardware. A fluent language plan is not a verified motion command.

Intermediate 1 Q&A

286 **How should language planning connect to robot control?**

Intermediate

Translate high-level goals into bounded capabilities with validated preconditions and feedback from the physical state. Keep fast control loops and safety constraints in suitable deterministic or specialized controllers. Replan when observations invalidate assumptions. The language model should not directly bypass collision limits or emergency-stop mechanisms.

Advanced 1 Q&A

287 **How do you evaluate transfer from simulation to a real robot?**

Advanced

Test variation in sensing, dynamics, object properties, timing, and environment conditions, then validate incrementally on hardware under controlled constraints. Measure success and unsafe behavior separately. Simulation coverage reduces risk but cannot prove physical performance. Keep emergency intervention and clear operating limits during real-world evaluation.

Edge deployment

Basic 1 Q&A

288 **Why might an agent run at the edge?**

Basic

Local execution can reduce network dependence, improve response time, and limit which data leaves a device. It also faces memory, compute, power, and update constraints. Decide which tasks truly need local inference and which can use approved remote services. Edge deployment does not automatically guarantee privacy.

Intermediate 1 Q&A

289 **How do you split work between a device and a cloud agent?**

Intermediate

Place latency-critical or sensitive processing locally when appropriate and send only permitted information to remote capabilities. Define behavior when connectivity is unavailable and reconcile state on reconnection. Keep authorization and version compatibility explicit. A hybrid design should degrade predictably rather than silently abandoning local safety checks.

Advanced 1 Q&A

290 **How do you update a fleet of edge agents reliably?**

Advanced

Use authenticated versioned packages, staged rollout, compatibility checks, and rollback or recovery paths appropriate to the device. Track hardware variation, offline periods, and partially applied updates. Separate model and policy changes when necessary. Evaluate the complete fleet state, not just whether one connected test device accepted the update.

Uncertainty and safety

Basic 1 Q&A

291 **Why does sensor uncertainty matter for agent actions?**

Basic

A system may misidentify an object, location, or person, making an otherwise valid action inappropriate. Preserve uncertainty and require additional observation when confidence is insufficient for the action. Use constraints tied to real physical risk. Do not interpret a model's confident wording as calibrated sensor certainty.

Intermediate 1 Q&A

292 **What is a safe fallback for an embodied agent?**

Intermediate

A fallback is a predefined behavior that reduces risk when perception, planning, or connectivity fails, such as stopping motion or handing control to an operator. The correct behavior depends on the system and environment. Implement it in the control stack and test triggering conditions. A language instruction to be safe is insufficient.

Advanced 1 Q&A

293 **How do you define operating boundaries for a physical agent?**

Advanced

Specify permitted environments, speeds, loads, users, sensor conditions, and task classes using the relevant engineering and regulatory process. Monitor for conditions outside those limits and transition to a validated fallback. Validate changes before expanding scope. Success in one workspace does not establish suitability for a different physical environment.

CATEGORY 32 / 34

Economics, capacity and product strategy

3 topics · 9 questions

Unit economics

Basic 1 Q&A

294 **What is cost per successful task?**

Basic

It is the total attributable cost of attempting tasks divided by the number that meet a defined success criterion. Include failed attempts, retries, tool fees, and relevant execution costs. Specify the measurement period and task population. Cheap individual model calls can still produce an expensive service when many steps fail or repeat.

Intermediate 1 Q&A

295 **How do you estimate the cost of a new agent feature?**

Intermediate

Model task volume, calls per task, input and output lengths, tool usage, retries, and execution resources under realistic distributions. Include evaluation and operational overhead appropriate to the estimate. Validate assumptions with a small pilot. Report ranges rather than a single precise number derived from idealized prompts.

Advanced 1 Q&A

296 **How do you choose between more model compute and stronger product constraints?**

Advanced

Compare the improvement in task success against cost, latency, user friction, and risk. A constrained form or deterministic validation may resolve ambiguity more effectively than extra reasoning calls. Test alternatives under the same outcome criteria. Product design can change the problem rather than forcing the model to solve an unnecessarily broad one.

Capacity and demand

Basic 1 Q&A

297 **Why is registered-user count a weak capacity estimate?**

Basic

Only some users are active at once, and different tasks create very different model and tool loads. Capacity depends on peak concurrency, request distribution, run duration, and downstream limits. Measure those quantities and include burst behavior. A million accounts does not imply a million simultaneous agent runs.

Intermediate 1 Q&A

298 **How do you handle fairness between heavy and light users?**

Intermediate

Apply tenant or user quotas, weighted scheduling where appropriate, and concurrency limits that prevent long runs from monopolizing capacity. Make admission and cancellation behavior predictable. Measure waiting time by user segment. Fairness policy should reflect the product's commitments rather than emerge accidentally from queue ordering.

Advanced 1 Q&A

299 **How do you plan for cascading overload in a multi-step agent platform?**

Advanced

Model how retries and fan-out amplify demand across providers and tools. Use backpressure, bounded queues, deadlines, per-dependency concurrency limits, and load shedding. Test partial outages and recovery rather than only steady-state throughput. A degraded dependency should not trigger unlimited parallel retries that exhaust the rest of the platform.

Product fit

Basic 1 Q&A

300 **How do you decide whether a task should become an agent feature?**

Basic

Identify a recurring user problem, the required access, a measurable outcome, and why a simpler interface or workflow is insufficient. Estimate operational cost and consequences of failure. Validate with real users and a bounded prototype. Agent autonomy is a design choice, not a product benefit by itself.

Intermediate 1 Q&A

301 **How do you measure an agent's value to users?**

Intermediate

Measure task completion, saved effort, quality, trust, and user control using appropriate baselines and research methods. Track abandonment and corrections alongside adoption. Distinguish novelty-driven use from sustained utility. A feature can generate many interactions while creating more review work than it saves.

Advanced 1 Q&A

302 **When should you reduce an agent's scope after launch?**

Advanced

Reduce scope when evidence shows repeated failures, unacceptable intervention cost, poor economics, or risk outside the supported operating conditions. Identify which task slices still provide value and constrain access accordingly. Communicate supported behavior clearly and evaluate the narrower system. Fewer capabilities can produce a more dependable product.

CATEGORY 33 / 34

Open standards and agent interoperability

3 topics · 9 questions

Protocol boundaries

Basic 1 Q&A

303 **How is agent-to-agent communication different from tool calling?**

Basic

Tool calling usually requests a defined operation from a capability. Agent-to-agent communication can delegate a broader task with progress, negotiation, or artifacts. Boundaries vary by implementation. In both cases, define identity, permissions, schemas, timeouts, and completion criteria rather than relying on informal conversation alone.

Intermediate 1 Q&A

304 **What should an inter-agent task contract contain?**

Intermediate

Include the goal, authorized scope, inputs, expected outputs, deadlines, budget, status model, artifact format, and cancellation behavior. Specify which party owns external side effects and how failures are reported. Avoid assuming both agents share hidden conversation context. Explicit contracts make integration and evaluation possible.

Advanced 1 Q&A

305 **How do you establish trust between independently operated agents?**

Advanced

Authenticate services, verify the delegating identity and permitted scope, validate artifacts, and enforce policy at every receiving boundary. Limit transitive delegation and audit consequential actions. A protocol handshake proves compatibility or identity only within its guarantees; it does not establish that another agent's reasoning or evidence is correct.

Artifacts and schemas

Basic 1 Q&A

306 **Why should agents exchange structured artifacts?**

Basic

Structured artifacts define fields and status explicitly, reducing ambiguity in handoffs and allowing validation. Examples include a research evidence bundle or a code-change report. Preserve references to original sources and distinguish unknown from absent data. Human-readable summaries remain useful but should not be the only machine contract for important workflows.

Intermediate 1 Q&A

307 **How do you evolve schemas used by multiple agents?**

Intermediate

Use version identifiers, backward-compatible additions where feasible, and explicit migrations for breaking changes. Define required fields and unknown-field behavior. Validate both producers and consumers with shared contract tests. Do not silently change the meaning of an existing field while retaining its name.

Advanced 1 Q&A

308 **How do you preserve provenance across a chain of delegations?**

Advanced

Carry stable task, source, and artifact identifiers with timestamps, authorship or service identity, and transformations performed. Preserve links to supporting evidence and permission scope. Validate provenance claims where they affect trust. A final summary should not erase which parts were observed, inferred, or supplied by an untrusted participant.

Interoperability operations

Basic 1 Q&A

309 **What is capability negotiation between agent systems?**

Basic

It is a process for discovering which features, versions, and operations each participant supports. Use the negotiated result to select compatible behavior and reject unsupported requests clearly. Discovery should be permission-scoped. Advertising a capability does not mean every caller is authorized to use it.

Intermediate 1 Q&A

310 **How do you handle cancellation across delegated agents?**

Intermediate

Define a cancellation identifier and state transition, propagate the request to active workers, and report which operations stopped versus already completed. Use deadlines and cleanup rules for abandoned work. Cancellation is best-effort for external actions already accepted by another service, so reconcile results rather than promising universal rollback.

Advanced 1 Q&A

311 **How do you test interoperability without assuming one vendor's behavior?**

Advanced

Build protocol conformance and contract tests covering schemas, version negotiation, errors, timeouts, retries, cancellation, and permission denial. Run against multiple independent implementations when possible. Include malformed and unexpected inputs. Successful message exchange is only the first step; verify end-to-end task state and protected boundaries.

CATEGORY 34 / 34

Advanced architecture and frontier concepts

3 topics · 9 questions

World models and simulation

Basic 1 Q&A

312 **What is a world model in an agent discussion?**

Basic

The term often refers to a representation that predicts aspects of an environment and how actions may change it. It may be learned or explicitly coded, and usage varies across research and products. Distinguish a model's plausible imagined outcome from an observed external result. Define the term before making capability claims.

Intermediate 1 Q&A

313 **How can simulation help an agent choose actions?**

Intermediate

A simulator can estimate candidate outcomes under specified assumptions, helping compare plans before execution. Validate how well it represents the real task and account for uncertainty. Keep simulated actions separate from real operations. A plan that succeeds in an inaccurate simulator may fail systematically in deployment.

Advanced 1 Q&A

314 **How do you prevent an agent from exploiting simulator weaknesses?**

Advanced

Use held-out environments, varied dynamics, independent outcome checks, and tests outside the optimization simulator. Inspect suspicious shortcuts and compare predicted with observed real-world behavior. Limit deployment based on evidence about transfer. High simulation reward alone is insufficient for a claim of general real-world competence.

Search and deliberation

Basic 1 Q&A

315 **What is planning search in an agent system?**

Basic

Planning search explores alternative action sequences or candidate solutions before choosing one. It can be implemented with code, model calls, or both. Search is useful only if candidates can be evaluated meaningfully. Bound compute and compare against a simpler direct approach.

Intermediate 1 Q&A

316 **When can extra deliberation make an agent worse?**

Intermediate

It can add latency, cost, contradictory assumptions, and opportunities to overfit a flawed evaluator. A correct simple answer may be revised into an incorrect elaborate one. Use stopping criteria and evidence-based evaluation. More tokens or critique rounds should earn their place through measured task improvements.

Advanced 1 Q&A

317 **How do you compare search-based planning with reactive agent loops?**

Advanced

Evaluate both on tasks with varying horizon, observability, and recovery cost, under comparable total budgets. Measure successful outcomes, constraint violations, compute, and adaptation to unexpected events. Search may help when future consequences can be modeled; reactive loops may suit rapidly changing conditions. Avoid declaring one architecture universally superior.

Self-modification and scope

Basic 1 Q&A

318 **What is the difference between self-reflection and self-modification?**

Basic

Self-reflection produces feedback or revised output within a run. Self-modification changes persistent instructions, code, tools, or model parameters. The latter can affect future tasks and needs explicit controls. A generated suggestion to improve a system is not evidence that a safe improvement was applied.

Intermediate 1 Q&A

319 How should an agent propose changes to its own tools?

Intermediate

Produce a reviewable patch or specification, explain the observed problem, and include validation evidence. Test changes in an isolated environment before promotion. Preserve permission boundaries and version compatibility. The agent proposing a change should not automatically gain authority to weaken the controls that evaluate or authorize it.

Advanced 1 Q&A

320 How do you evaluate claims of general-purpose autonomous agents?

Advanced

Ask for a defined task distribution, operating constraints, failure reporting, intervention rates, and independent outcome checks. Examine contamination, hidden human assistance, and selection of demonstrations. Compare against baselines under equal budgets. Broad marketing language does not establish reliable performance across unseen tasks and environments.

KEEP LEARNING

Primary references and editorial notes

The uploaded bhavs.ai carousel is the source for the 72 labeled question prompts. These links are further reading for checking implementation details; this edition is an original explanatory guide, not a live-verified survey of every product or protocol version.

- Model Context Protocol documentation — primitives, clients, servers, and protocol capabilities.
- OpenAI function calling and structured outputs — tool execution and schema contracts.
- Anthropic: Building effective agents — workflow and agent design patterns.
- OWASP GenAI Security Project — application security risks and defenses.
- NIST AI Risk Management Framework — governance and risk management.
- Evaluating Large Language Models Trained on Code — pass@k evaluation context.

Provider behavior, pricing, model capabilities, SDKs, and MCP revisions change. Verify supported features against the exact version used in your project. “Basic,” “Intermediate,” and “Advanced” describe the expected depth of an answer rather than a universal hiring standard.

Agentic Atlas Q&A · Edition 02 · Updated 4 November 2026
Expandable structure: category → topic → difficulty → question → answer. Keep the accompanying JSON file as the editable content source.